

DIGITAL NOTICE BOARD AND ALERT MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

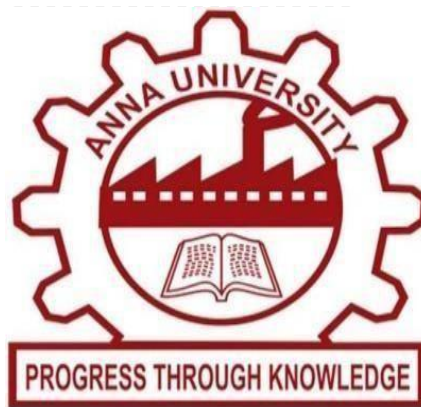
KIRTHICK ROSHAN S 810022104116

BHARANIDHARAN J T 810022104302

in partial fulfillment for the award of the degree

of

**BACHELOR OF ENGINEERING
IN
COMPUTER SCIENCE AND ENGINEERING**



**UNIVERSITY COLLEGE OF ENGINEERING -BIT CAMPUS
TIRUCHIRAPPALLI – 620024**

ANNA UNIVERSITY : CHENNAI 600 025

MAY 2026

ANNA UNIVERSITY : CHENNAI 600 025

BONAFIDE CERTIFICATE

Certified that this project report “**DIGITAL NOTICE BOARD AND ALERT MANAGEMENT SYSTEM**” is the bonafide work of “**KIRTHICK ROSHAN S (810022104116), BHARANIDHARAN J T (810022104302)**” who carried out the project work under my supervision.

SIGNATURE

Dr. G. ANNAPOORANI

HEAD OF THE DEPARTMENT

Assistant Professor (Sl. Gr)

Department of CSE/IT

University College of Engineering,

BIT Campus, Anna University

Tiruchirappalli – 620 024.

SIGNATURE

Dr. C. USHARANI

SUPERVISOR

Assistant Professor (Sl. Gr)

Department of CSE

University College of Engineering,

BIT Campus, Anna University

Tiruchirappalli – 620 024.

Submitted for the VIVA-VOCE to be held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

We hereby declare that the work entitled “**DIGITAL NOTICE BOARD AND ALERT MANAGEMENT SYSTEM**” is submitted in partial fulfilment of the requirements for the award of the degree in B.E. – Computer Science and Engineering in University College of Engineering, BIT Campus, Anna University, Tiruchirappalli. It is the record of my own work carried out during the academic year 2025-2026 under the supervision and guidance of **Dr. C. Usharani** Assistant Professor (Sl. Gr), Department of Computer Science and Engineering, University College of Engineering, BIT Campus, Anna University, Tiruchirappalli. The extent and source of information are derived from the existing literature and have been indicated through the dissertation at the appropriate places.

KIRTHICK ROSHAN S (810022104116)

BHARANIDHARAN J T (810022104302)

I certify that the declaration made above by the candidate is true.

Signature of the Guide

Dr. C. USHARANI

Assistant Professor (Sl. Gr)

Department of CSE

University College of Engineering,

BIT Campus, Anna University,

Tiruchirappalli – 620024.

ACKNOWLEDGEMENT

We would like to thank our honorable Dean **Dr. T. SENTHILKUMAR**, Professor, for having provided us with all the required facilities to complete our project without hurdles.

We would also like to express our sincere thanks to **Dr. G. ANNAPOORANI**, Assistant Professor (Sl. Gr), Head of the Department of CSE/IT, for her valuable guidance, suggestions and constant encouragement paved the way for the successful completion of this project work.

We would like to thank our Project Coordinator **Dr. C. USHARANI** Assistant Professor (Sl. Gr), Department of Computer Science and Engineering, for her kind support.

We would like to thank and express our deep sense of gratitude to our project guide **Dr. C. USHARANI** Assistant Professor (Sl. Gr), Assistant Professor, Department of Computer Science and Engineering, for his valuable guidance throughout the project.

We also extend our thanks to all other teaching and non-teaching staff for their encouragement and support. We thank our beloved parents and friends for their full support in the moral development of this project.

ABSTRACT

Digital Notice Board and Alert Management Systems are essential applications in modern educational institutions that require continuous availability, reliable information delivery, and efficient communication management. Traditional methods of sharing information, such as physical notice boards and scattered digital platforms, are often inefficient, time-consuming, and prone to delays or missed announcements. These limitations create communication gaps between departments and students, making it difficult to ensure that important academic information reaches the intended audience on time. The need for a centralized, automated, and intelligent communication system has therefore become increasingly important. This project designs and implements a smart Digital Notice Board and Alert Management System that enhances communication efficiency through automation and real-time information delivery. The system provides a centralized platform where departments and administrators can create, manage, and publish notices with multimedia attachments, while students can access personalized and prioritized updates through an interactive dashboard. The application integrates structured workflows such as notice approval mechanisms, role-based access control, and engagement tracking to ensure secure and organized communication within the institution. The system further incorporates machine learning techniques to improve functionality and user experience. Algorithms such as LightGBM are used for notice ranking, DistilBERT for text understanding and smart search, Cox model for deadline risk prediction, and recommendation models for personalized content delivery. These intelligent components ensure that important notices are highlighted, deadlines are not missed, and users receive relevant information based on their behavior and academic profile. Real-time notifications and email alerts are also integrated to provide timely updates and reminders. The application is implemented as a full-stack web system using modern technologies, including a React-based frontend for an interactive user interface, a Django backend for processing and API management, and a relational database for secure data storage. The system is designed to be scalable, responsive, and accessible across multiple devices. By combining automation, personalization, and real-time communication, the Digital Notice Board and Alert Management System significantly improves transparency, efficiency, and user engagement, providing a practical and reliable solution for modern campus communication.

சுருக்கம்

டிஜிட்டல் நோட்டீஸ் போர்டு மற்றும் அலர்ட் மேலாண்மை அமைப்புகள் இன்றைய கல்வி நிறுவனங்களில் முக்கியமான தகவல் பரிமாற்ற கருவிகளாக வளர்ந்து வருகின்றன. பாரம்பரிய நோட்டீஸ் போர்டுகள் மற்றும் சிதறிய டிஜிட்டல் தளங்கள் தகவலை நேரத்தில் வழங்க முடியாததால், மாணவர்கள் மற்றும் துறைகள் இடையே தகவல் பரிமாற்றத்தில் இடைவெளிகள் உருவாகின்றன. இந்த சிக்கல்களை சரிசெய்ய மையப்படுத்தப்பட்ட, தானியக்க மற்றும் புத்திசாலியான தகவல் பரிமாற்ற அமைப்பு தேவையானதாகியுள்ளது. இந்த திட்டம் ஒரு நவீன டிஜிட்டல் நோட்டீஸ் போர்டு மற்றும் அலர்ட் மேலாண்மை அமைப்பை வடிவமைத்து செயல்படுத்துகிறது. இந்த அமைப்பு மூலம் நிர்வாகிகள் மற்றும் துறைகள் நோட்டீஸ்களை எளிதாக உருவாக்கி, நிர்வகித்து, வெளியிட முடியும். மாணவர்கள் தனிப்பயனாக்கப்பட்ட தகவல்களை ஒரு இன்டர்அக்டிவ் டாஷ்போர்டின் மூலம் உடனுக்குடன் பெற முடியும். மேலும், Role-Based Access Control, நோட்டீஸ் அங்கீகார முறை மற்றும் பயனர் ஈடுபாட்டு கண்காணிப்பு போன்ற அம்சங்கள் பாதுகாப்பான மற்றும் ஒழுங்குபடுத்தப்பட்ட தகவல் பரிமாற்றத்தை உறுதிப்படுத்துகின்றன. இந்த அமைப்பு இயந்திரக் கற்றல் நுட்பங்களையும் பயன்படுத்துகிறது. LightGBM மூலம் முக்கிய நோட்டீஸ்கள் முன்னுரிமைப்படுத்தப்படுகின்றன; DistilBERT மூலம் உரை புரிதல் மற்றும் சாமர்த்தியமான தேடல் செய்யப்படுகிறது; Cox மாடல் மூலம் காலக்கெடு அபாயம் கணிக்கப்படுகிறது; பரிந்துரை மாடல்கள் மூலம் பயனாளர்களுக்கு தொடர்புடைய தகவல்கள் வழங்கப்படுகின்றன. இதனால் பயனாளர்கள் முக்கிய அறிவிப்புகளை தவறவிடாமல், தேவையான தகவல்களை சரியான நேரத்தில் பெற முடிகிறது. மேலும், real-time notifications மற்றும் email alerts உடனடி தகவல் பரிமாற்றத்தை உறுதிப்படுத்துகின்றன. இந்த பயன்பாடு React முன்புறம், Django பின்புறம் மற்றும் தொடர்புடைய தரவுத்தளத்தை பயன்படுத்தி உருவாக்கப்பட்டுள்ளது. இது பல சாதனங்களில் அணுகக்கூடியதாகவும், விரிவாக்கத்திற்கும் ஏற்றதாகவும் வடிவமைக்கப்பட்டுள்ளது. மொத்தத்தில், இந்த டிஜிட்டல் நோட்டீஸ் போர்டு மற்றும் அலர்ட் மேலாண்மை அமைப்பு கல்வி நிறுவனங்களில் தகவல் பரிமாற்றத்தை மேம்படுத்தும் ஒரு திறமையான, நம்பகமான மற்றும் பயனுள்ள தீர்வாக அமைகிறது.

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	ABSTRACT	v
	LIST OF TABLES	ix
	LIST OF FIGURES	x
	LIST OF ABBREVIATIONS	xi
	LIST OF ALGORITHMS	xii
1	INTRODUCTION	1
	1.1 Background	1
	1.2 Problem Statement	1
	1.3 Objectives	2
	1.4 Scope of The Project	2
2	LITERATURE SURVEY	4
	2.1 Introduction to Literature Review	4
	2.2 Review of Existing System of the Papers	5
	2.3 Comparative Analysis Existing Systems	7
3	PROPOSED SYSTEM	10
	3.1 Existing System	10
	3.2 Proposed System	11
	3.3 System Architecture	12
	3.4 Methodology	15
	3.4.1 User Authentication and Login Process	16
	3.4.2 User Profile Retrieval	16
	3.4.3 Retrieval of Notices from Database	17
	3.4.4 Data Preprocessing	17
	3.4.5 Application of Machine Learning Algorithms	17
	3.4.6 Output Generation	18

3.5 Algorithms	18
3.5.1 LightGBM	18
3.5.2 DistilBERT	19
3.5.3 Proportional Hazard Model	20
3.5.4 Matrix Factorization	22
3.6 Data Flow Diagram	23
3.7 Module Description	24
4 IMPLEMENTATION AND RESULTS	27
4.1 Tools & Technologies	27
4.1.1 Hardware Requirements	27
4.1.2 Software Requirements	27
4.2 System Implementation	28
4.2.1 Frontend Implementation	29
4.2.2 Backend Implementation	30
4.2.3 Database Implementation	31
4.2.4 Database Design	31
4.2.5 Email Automation System	36
4.2.6 Timetable Generation	36
4.3 Working Model	36
4.4 Results and Output	44
4.5 Performance Analysis	44
5 CONCLUSION	47
5.1 Conclusion	47
5.2 Future Enhancements	47
5.3 Real World Extensions	48
APPENDIX	49
Appendix A: System Configuration	49
Appendix-B: Database Schema	50
Appendix-C: REST API Reference	51
REFERENCES	53

LIST OF TABLES

TABLE NO	TITLE	PAGE NO
1	Comparative Analysis of Existing Systems	9
2	Hardware Requirements	27
3	Software Requirements	27
4	Key backend modules	31
5	User Table	32
6	Student profile Table	32
7	Department Table	33
8	Notice Table	33
9	Notice Attachment Table	34
10	User Notification Table	34
11	Timetable Batch Table	35
12	Timetable Entry Table	35
13	Interaction Log table	36

LIST OF FIGURES

FIGURE NO	TITLE	PAGE NO
1	System Architecture	12
2	Methodology	15
3	Data Flow Diagram	23
4	Login Page	37
5	Student Login	37
6	Student Dashboard	38
7	Student Notice	38
8	Notice Popup	39
9	Student Notification	39
10	Searching	40
11	Set Reminder	40
12	Downloading Attachment Documents	41
13	Profile Page	41
14	Email Notification Received	42
15	Admin Dashboard	43
16	Department Dashboard	43
17	Create Notice	44
18	Accuracy	45
19	Efficiency	46

LIST OF ABBREVIATIONS

ABBREVIATION	FULL FORM
AI	Artificial Intelligence
ML	Machine Learning
NLP	Natural Language Processing
API	Application Programming Interface
UI	User Interface
UX	User Experience
DB	Database
SQL	Structured Query Language
HTTP	Hyper Text Transfer Protocol
HTTPS	Hyper Text Transfer Protocol Secure
JWT	JSON Web Token
RBAC	Role-Based Access Control
SMTP	Simple Mail Transfer Protocol
PDF	Portable Document Format
CSV	Comma-Separated Values
IDE	Integrated Development Environment
OS	Operating System
IOT	Internet of Things
SVD	Singular Value Decomposition
HOD	Head of Department

LIST OF ALGORITHMS

ALGO. NO	ALGORITHM NAME	PAGE NO.
1	LightGBM	18
2	DistilBERT	19
3	Proportional Hazard Model	20
4	Matrix Factorization	22

CHAPTER 1

INTRODUCTION

1.1 Background

The rapid growth of information and communication technology has significantly improved how educational institutions manage and share information. In colleges and universities, effective communication is essential to keep students, faculty, and administrators informed about academic activities, events, exams, placements, and other important updates. However, traditional methods such as physical notice boards, emails, and messaging platforms often result in delayed communication, missed announcements, and difficulty in tracking important information. Students may overlook notices due to lack of prioritization or information overload, which affects their academic performance and awareness.

To address these issues, the Digital Notice Board and Alert Management System is proposed as a centralized and intelligent platform that automates notice management and ensures timely communication. The system provides features such as notice prioritization, personalized notifications, real-time alerts, and email reminders. It also supports role-based access for students, departments, and administrators, enabling structured workflows and secure communication. By integrating automation and smart technologies, the system improves efficiency, reduces manual effort, and ensures that important information reaches the right users at the right time, enhancing overall communication within educational institutions.

1.2 Problem Statement

Modern educational institutions require fast, reliable, and centralized communication systems to distribute academic and administrative information. Traditional notice boards rely on manual updates, printed circulars, and informal communication channels, which often lead to delayed information delivery, missed announcements, and lack of proper tracking. Important notices such as exam schedules, placement updates, deadlines, and event announcements may not reach all students on time, affecting academic performance and student participation.

In many colleges, existing digital communication methods such as email groups or messaging platforms are unorganized and not personalized for each department or student year. Notifications are often sent in bulk without prioritization, causing students to overlook critical updates. There is also no proper mechanism to track whether a student has received, viewed, or acknowledged a notice. This creates communication gaps between departments and students and increases the risk of missing important deadlines.

Another major challenge is the absence of an approval workflow for notices. In most cases, notices are shared without proper verification, leading to confusion, misinformation, or duplicate announcements.

Additionally, there is no intelligent system to categorize notices based on priority, deadlines, or student relevance. Students from different departments and academic years receive the same notifications, even when the information is not applicable to them.

Therefore, there is a strong need for a centralized, automated, and intelligent Digital Notice Board and Alert Management System that ensures secure notice publishing, approval-based workflows, personalized notifications, and deadline-aware alerts. The system should provide real-time communication, track student interactions, and deliver priority-based notifications to the right audience at the right time, improving communication efficiency across departments and enhancing the overall academic experience.

1.3 Objectives

The main objective of the Digital Notice Board and Alert Management System is to develop a smart, centralized, and real-time platform that improves the way announcements and alerts are created, managed, and delivered within educational institutions and organizations. The system aims to replace traditional paper-based notice boards with an efficient digital solution that ensures timely communication, reduces manual effort, and enhances accessibility.

The objective of the proposed system is to design and develop a centralized web-based platform for creating, managing, and displaying notices digitally. The system enables authorized administrators to post announcements, alerts, and event notifications instantly from any location, thereby improving communication efficiency. It provides real-time alert delivery to users through dashboards and notification systems, ensuring that important updates are received without delay. Additionally, the system incorporates role-based access control for administrators, staff, and students to maintain data security and privacy.

It aims to reduce paper usage and eliminate manual notice distribution, thereby promoting an eco-friendly approach to communication. The platform also supports scheduling and automatic expiration of notices to ensure that information remains relevant and up to date. Furthermore, it enhances the visibility and accessibility of important information, allowing users to access notices anytime and from anywhere. Finally, the system is designed to be scalable and user-friendly, making it suitable for adoption in schools, colleges, and small organizations.

1.4 Scope of the Project

The scope of the proposed **Digital Notice Board and Alert Management System** focuses on developing a centralized, web-based platform designed to improve communication within educational institutions. The system is intended to be used by students, staff, department administrators, Heads of Departments (HODs), and institutional administrators. It provides a structured and efficient way to create, manage, distribute, and monitor notices in real time. The platform eliminates the need for traditional notice boards and scattered communication channels by offering a unified digital solution.

The system supports multiple categories of notices such as academic updates, examination schedules, placement information, workshops, scholarships, events, and holidays. It ensures that users receive only relevant information based on their role, department, and preferences. Additionally, the system introduces intelligent features such as notice prioritization, personalized recommendations, and deadline-based alerts to enhance user experience and reduce the chances of missing important information.

The application is designed to operate primarily within a single college or university environment; however, it is scalable and can be extended to support multiple institutions in the future. It also includes advanced modules such as staff-specific notices, HOD approval workflows, and automated timetable generation, making it a comprehensive communication and management system.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction of Literature Review

The rapid advancement of digital technologies has significantly transformed communication methods within educational institutions. Traditional notice boards, which rely on manual posting of information, are gradually being replaced by web-based and automated systems. Digital notice board systems have emerged as an efficient solution for managing and disseminating information such as academic announcements, examination schedules, event updates, and administrative notices. These systems enable centralized communication, reduce manual effort, and ensure that information reaches students in a timely and organized manner.

However, as the volume of information increases, managing and prioritizing notices becomes a challenging task. Many existing digital notice systems focus primarily on displaying information but lack intelligent features such as personalization, prioritization, and real-time alert mechanisms. As a result, students may miss important updates due to information overload or lack of proper notification systems. This highlights the need for an advanced system that not only displays notices but also ensures that relevant information is delivered effectively to the right users.

A literature review plays a crucial role in understanding the current state of research and development in this domain. It provides a structured analysis of existing systems, technologies, and methodologies used in digital notice management and alert systems. By examining previous research works, it becomes possible to identify key features, limitations, and gaps in current approaches. In the context of this project, the review focuses on web-based notice systems, real-time notification frameworks, and intelligent information filtering techniques.

Recent studies indicate that while digital notice systems improve accessibility and communication efficiency, they often lack advanced capabilities such as machine learning-based prioritization, deadline prediction, and user-specific recommendations. Additionally, many systems do not provide integrated alert mechanisms like email notifications or real-time updates, which are essential for ensuring that users do not miss critical information. Therefore, there is a growing need for a smart and adaptive system that integrates modern technologies to enhance user experience and communication effectiveness.

The proposed Digital Notice Board and Alert Management System aims to address these limitations by incorporating intelligent features such as notice ranking, personalized recommendations, real-time notifications, and automated alert mechanisms. This approach ensures efficient information delivery, improves user engagement, and supports better academic management within educational institutions.

2.2 Review of Existing Systems and Research Papers

Paper 1: Ramakrishna and Karthik (2024) — IoT-Based Smart Digital Notice Board with Multimedia Support

Ramakrishna and Karthik proposed a smart digital notice board system that utilizes web-based platforms along with IoT technologies to deliver real-time updates and multimedia content. The system allows administrators to publish notices dynamically, including text, images, and videos, improving communication efficiency. It reduces dependency on traditional paper-based systems and supports centralized control. The platform enhances accessibility by enabling users to view notices from multiple devices such as mobile phones and desktops.

Limitations: *The system requires a high initial setup cost due to IoT hardware integration. It depends heavily on continuous internet connectivity for proper functioning. The architecture becomes complex when scaling across multiple departments. It also lacks personalization and intelligent filtering of notices for individual users.*

Paper 2: Islam, Rahman, and Hossain (2025) —Raspberry Pi-Based IoT Digital Notice Board System

Islam et al. developed an IoT-based digital notice board system using Raspberry Pi, allowing administrators to remotely post notices through a web interface. The system provides real-time updates and reduces manual effort in maintaining physical boards. It improves communication efficiency within institutions and ensures instant message delivery. The system also supports basic multimedia display, making it more flexible than traditional methods.

Limitations: *The system lacks advanced features such as prioritization and ranking of notices. It does not provide personalized content for different users. There is no integration of intelligent notification or alert systems. Additionally, the system depends on hardware components, increasing maintenance complexity.*

Paper 3: Sharma and Patel (2025) —Mobile-Based IoT Notice Board for Remote Communication

This research introduces a mobile-based IoT notice board system where users can update and manage notices remotely using mobile applications. The system enhances communication by providing real-time updates and easy access to notices. It reduces dependency on physical notice boards and improves user convenience. The integration of mobile technology makes the system more flexible and accessible.

Limitations: *The system focuses mainly on message transmission without advanced processing. It does not include intelligent filtering or prioritization of notices. There is no support for recommendation systems or user personalization. The system lacks analytics and tracking features for user engagement.*

Paper 4: InfoVista Research Group (2025) —Cloud-Integrated Smart Notice Board with Voice Announcement

The InfoVista system integrates IOT and cloud computing technologies to provide real-time updates and voice announcements. It enhances accessibility by allowing users to receive information through multiple channels. The system supports centralized management and improves communication efficiency across campus environments. It also allows multimedia content display, making it interactive and user-friendly.

***Limitations:** The system has a complex architecture that requires high technical expertise to manage. It involves higher implementation and maintenance costs. The system does not include personalization features for different users. It also lacks intelligent ranking and predictive capabilities.*

Paper 5: Gupta and Verma (2020) —Wireless Digital Notice Board Using Raspberry Pi

Gupta et al. proposed a wireless digital notice board using Raspberry Pi, enabling remote posting of notices. The system supports multiple formats such as text, images, and PDF files, improving flexibility. It reduces manual effort and enhances communication efficiency in institutions. The system provides a simple and effective solution for digital notice management.

***Limitations:** The system does not include real-time alert mechanisms for users. It lacks intelligent ranking of notices based on priority. There is no personalization or recommendation feature. The system does not support predictive analysis or deadline tracking.*

Paper 6: Kumar and Singh (2024) —Bluetooth and Wi-Fi Enabled Wireless LED Notice Board

This research focuses on a wireless LED-based notice board system using Bluetooth and Wi-Fi technologies. It allows administrators to update content remotely and display messages efficiently. The system is cost-effective and easy to implement for small-scale applications. It improves visibility and accessibility compared to traditional boards.

***Limitations:** The system has limited scalability for large institutions. It lacks centralized web-based management capabilities. There are no advanced features such as personalization or intelligent filtering. The system also does not support multimedia content effectively.*

Paper 7: Singh, Rao, and Mehta (2024) —Review of Smart Digital Notice Board Technologies

This review paper analyzes the evolution of digital notice boards and their role in modern communication systems. It highlights the benefits such as efficiency, cost reduction, and real-time updates. The study compares various technologies used in digital notice systems. It provides insights into current trends and future developments in the field.

Limitations: *The paper does not provide practical implementation details. It lacks experimental validation or system development. There is no integration of machine learning or intelligent features. The study is theoretical and does not address real-world challenges.*

Paper 8: Reddy and Kumar (2024) —IoT and Android-Based Digital Campus Notice System

This system combines IoT technology with Android applications to provide real-time notice updates. It allows users to access notices remotely through mobile devices. The system improves communication efficiency and reduces paper usage. It provides a user-friendly interface for managing and viewing notices.

Limitations: *The system does not include machine learning-based features. It lacks personalization and recommendation capabilities. There is no intelligent ranking of notices based on priority. The system also does not provide predictive alerts for deadlines.*

Paper 9: Patel and Shah (2023) —Multimedia Smart Classroom Notice Board System

This paper presents a digital notice board system designed for classroom environments. It supports multimedia content such as images, videos, and audio for effective communication. The system enhances the learning environment by providing dynamic content display. It improves engagement compared to traditional notice boards.

Limitations: *The system focuses only on content display functionality. It does not support intelligent user interaction or personalization. There is no ranking or filtering mechanism for notices. The system lacks analytics and predictive capabilities.*

Paper 10: Ahmed and Khan (2022) —GSM-Based Wireless Digital Notice Display System

This system uses GSM technology to send messages directly to a digital display board. It enables wireless communication without requiring internet connectivity. The system is simple and cost-effective for basic applications. It reduces manual effort in updating notice boards.

Limitations: *The system supports only text-based messages. It lacks multimedia support such as images and videos. There is no scalability for large institutions. The system does not include intelligent features such as ranking, personalization, or alerts.*

2.3 Comparative Analysis of Existing Systems

The reviewed systems can be broadly grouped into four major categories based on their functionality and technological approach. First, IoT-based notice board systems (Ramakrishna, Islam, Gupta, and others) focus on real-time display and remote updating using hardware devices such as Raspberry Pi and LED boards. These systems improve communication efficiency but mainly operate at the hardware level and lack intelligent processing capabilities. Second, web-based notice systems provide centralized platforms for managing notices, allowing administrators to publish and manage content through web interfaces.

While these systems improve accessibility and organization, they lack advanced features such as real-time alerts, prioritization, and personalization. Third, mobile and cloud-integrated systems enhance accessibility by enabling users to receive updates on mobile devices, but they still focus primarily on information delivery rather than intelligent processing. Finally, review and survey papers provide insights into trends and technologies but do not offer practical implementations or advanced system integration.

From this classification, it is evident that most existing systems operate primarily at the display and communication layer, focusing on delivering notices rather than enhancing how users interact with or prioritize them. These systems do not incorporate intelligent mechanisms to analyze user behavior, predict deadlines, or recommend relevant content. As a result, users often receive the same information regardless of their role, interests, or urgency of the notice.

The comparative analysis presented in Table 2.1 evaluates these systems based on four key parameters: technology used, system capability, level of intelligence, and limitations. Systems that rely only on IoT or basic web technologies are effective for displaying notices but fail to provide real-time alerts, ranking, and personalization. Similarly, systems that support remote access improve usability but still lack predictive and adaptive features. None of the existing systems provide a unified solution that integrates real-time communication, intelligent prioritization, personalized recommendations, and predictive alert mechanisms within a single platform.

A significant limitation identified in current systems is the absence of pre-delivery prioritization and personalization. Most systems display notices after they are published without evaluating their importance or relevance to individual users. This leads to information overload, where critical notices may be overlooked. Additionally, there is no mechanism to predict whether a user is likely to miss a deadline or require reminders. Existing notification systems are either manual or rule-based, lacking the ability to adapt dynamically based on user behavior.

Another important gap is the lack of integration between data analytics and user interaction tracking. While some systems record basic data, they do not utilize this information to improve system performance or user experience. There is also no support for intelligent search mechanisms, as most systems rely on keyword-based search rather than context-aware understanding.

The proposed Digital Notice Board and Alert Management System addresses these limitations by introducing an integrated and intelligent framework. The system combines machine learning algorithms such as LightGBM for ranking, DistilBERT for semantic understanding, the Cox model for risk prediction, and matrix factorization for recommendations. This ensures that notices are not only delivered but also prioritized, personalized, and predicted based on user behavior and system data.

Furthermore, the system introduces a real-time notification and alert mechanism, ensuring that users receive timely updates and reminders. Unlike existing systems, the proposed solution

dynamically adjusts content visibility and importance based on user interactions, deadlines, and relevance. It also integrates seamlessly with modern web technologies, providing a scalable and flexible architecture suitable for educational institutions.

Table 1: Comparative Analysis of Existing Systems table

No	System	Technology	Advantages	Limitations
1	IoT Notice Board	Raspberry Pi	Real-time update	No personalization
2	Wireless LED	Bluetooth/WiFi	Low cost	Not scalable
3	Web Notice System	PHP/MySQL	Centralized	No alerts
4	GSM Board	GSM	No internet needed	Text only
5	IoT + Mobile	App + IoT	Mobile access	No ML features
6	Smart Campus	Cloud + IoT	Scalable	High cost
7	Review Paper	Survey	Trend analysis	No implementation
8	Smart Classroom	Multimedia	Interactive	No alerts
9	Basic Notification	Email	Simple alerts	No ranking
10	Proposed System	React + Django + ML	Smart, personalized	Slight complexity

CHAPTER 3

PROPOSED SYSTEM

3.1 Existing System

The existing notice management system in educational institutions mainly relies on traditional notice boards and basic digital platforms such as websites, emails, and messaging groups. Notices are usually posted manually, and students are expected to check these sources regularly to stay updated. This process is time-consuming and often leads to missed information due to the lack of a centralized and automated communication system.

Most existing systems provide only static content and do not support real-time updates or dynamic delivery of information. Communication is scattered across multiple platforms, making it difficult for students to track important announcements efficiently. Additionally, these systems lack advanced features such as personalization, intelligent filtering, and automated alerts, which reduces their overall effectiveness.

Limitations of Existing System

The existing system has several significant limitations that affect communication efficiency and user experience:

- **Lack of Centralization:** Information is distributed across multiple platforms such as notice boards, emails, and messaging apps, leading to confusion and inconsistency.
- **Manual Process:** Notices are posted and managed manually, increasing workload and chances of human error.
- **No Real-Time Updates:** Most systems do not provide instant updates, causing delays in information delivery.
- **No Personalization:** All users receive the same notices regardless of their department, year, or interests, resulting in irrelevant information.
- **No Priority-Based Display:** Important notices are not highlighted, making it difficult for students to identify urgent updates.
- **Poor Search Functionality:** Limited or basic search options make it hard to find specific notices quickly.
- **No Intelligent Filtering:** Users cannot filter notices effectively based on relevance or preferences.
- **Lack of Tracking and Analytics:** There is no system to monitor user engagement such as views, clicks, or downloads.

- **No Automated Alerts:** Students may miss deadlines due to the absence of reminder or alert systems.
- **Weak Security:** Lack of role-based access control can lead to unauthorized access or incorrect notice posting.
- **No Approval Workflow:** Notices can be published without proper verification, increasing the risk of errors.
- **Limited Multimedia Support:** Many systems support only text, with limited or no support for images, videos, or documents.
- **No Mobile Optimization:** Systems are not fully responsive, making them difficult to use on smartphones.
- **No Integration:** Lack of integration with modern systems such as LMS, ERP, or cloud platforms.
- **Scalability Issues:** Existing systems are not designed to handle large numbers of users or multiple departments efficiently.
- **High Chance of Missing Information:** Due to scattered communication and lack of alerts, important notices are often overlooked.

3.2 Proposed System

The proposed Digital Notice Board and Alert Management System is a web-based platform designed to improve communication within educational institutions. It replaces traditional notice boards with a centralized system where students, staff, and administrators can easily access notices. The system provides categorized viewing, real-time notifications, and role-based access to ensure efficient and secure information delivery.

It also includes intelligent features such as notice prioritization, personalized recommendations, and automated deadline alerts. Users can filter notices based on categories like academic updates, events, and placements. Additionally, the system supports timetable generation, helping departments manage academic schedules efficiently.

Advantages

- **Centralized Platform:** All notices are managed and accessed from a single system, eliminating confusion caused by multiple communication channels.
- **Automated Process:** Notice creation, publishing, and management are automated, reducing manual effort and minimizing human errors.
- **Real-Time Updates:** The system provides instant updates through real-time notifications, ensuring timely delivery of information.

- **Personalized Content:** Notices are delivered based on user profile such as department, academic year, and preferences, reducing irrelevant information.
- **Priority-Based Display:** Important and urgent notices are highlighted using intelligent ranking, making them easily identifiable.
- **Advanced Search Functionality:** Users can quickly find notices using powerful search options based on keywords, categories, and dates.
- **Intelligent Filtering:** The system allows filtering of notices based on relevance, category, and user preferences for better usability.
- **Automated Alerts and Reminders:** Notifications and reminders are automatically sent for important deadlines, reducing the chances of missing critical information.
- **Approval Workflow:** Notices go through a proper approval process (e.g., HOD/Admin approval) before publishing, ensuring accuracy and reliability.
- **Multimedia Support:** Supports various formats such as images, videos, PDFs, and documents for rich and detailed communication.
- **Mobile Responsiveness:** The system is fully responsive and works efficiently on smartphones, tablets, and desktops.

3.3 System Architecture

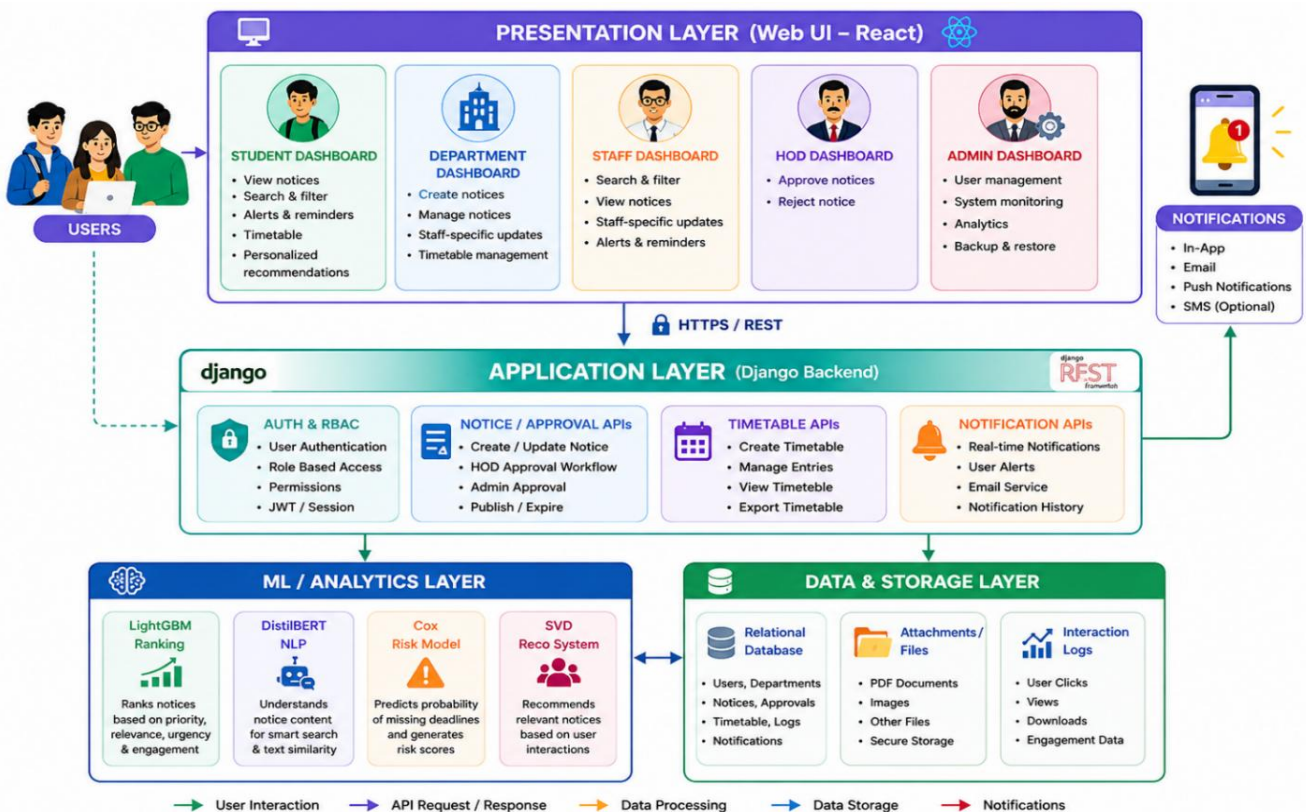


Figure 1: System Architecture

The proposed Digital Notice Board and Alert Management System follows a multi-layered architecture to ensure scalability, modularity, and efficient data processing. The system is divided into four main layers: Presentation Layer, Application Layer, ML/Analytics Layer, and Data & Storage Layer. Each layer performs specific functions and communicates with others through well-defined interfaces.

1. Presentation Layer (User Interface)

The Presentation Layer is responsible for user interaction and is developed using a React-based web user interface. It acts as the front-facing component of the system where users can access functionalities based on their assigned roles. The user interface is designed to be responsive, user-friendly, and accessible across multiple devices such as desktops, tablets, and mobile phones. This layer provides five different dashboards to cater to different user roles within the system.

The Student Dashboard is designed to provide students with easy access to academic information. Students can view all notices from their department and institution, search and filter notices based on categories such as exams, events, and placements, and receive alerts and reminders for important deadlines. Additionally, they can view generated timetables and receive personalized recommendations based on their interaction history. This ensures that students receive relevant and prioritized information, thereby reducing the chances of missing important updates.

The Department Dashboard is responsible for content creation and management. It allows department users to create and publish new notices, manage existing notices by editing, deleting, or updating them, provide staff-specific updates, and generate and manage timetables. This module acts as the primary content provider in the system. The Staff Dashboard focuses on information access and communication. Staff members can view notices relevant to them, search and filter notices, receive alerts and reminders, and access staff-specific updates. Unlike department users, staff members do not create notices but primarily consume departmental information.

The Head of Department (HOD) Module introduces an approval workflow within the system. The HOD is responsible for approving notices submitted by departments and rejecting them if necessary. This ensures that only validated and authorized information is published to students, thereby maintaining data integrity and institutional control. The Admin Dashboard provides full system control and monitoring capabilities. The administrator can manage users including students, staff, departments, and HODs, monitor system activities, view analytics and reports, and perform backup and restore operations. This ensures smooth functioning and centralized control of the system.

In addition, the presentation layer includes a notification system that supports in-app notifications, email alerts, and optional SMS alerts. This ensures real-time communication between the system and users.

2. Application Layer (Django Backend)

The Application Layer acts as the core processing unit of the system and is implemented using the Django framework with REST APIs. It handles all business logic, data processing, and communication between the frontend and backend components.

The Authentication and Role-Based Access Control (RBAC) module ensures secure access to the system. It includes user authentication mechanisms such as login and logout, role-based permissions for different users including students, staff, departments, HODs, and administrators, and authentication methods such as JWT or session-based authentication. Each user is granted access only to the features relevant to their role, ensuring both security and proper data access.

The Notice and Approval APIs manage the complete lifecycle of notices within the system. This includes the creation and updating of notices, the HOD approval or rejection workflow, optional admin-level approval, and the publishing and expiration of notices. This structured process ensures that all notices follow a controlled workflow before being visible to students.

The Timetable APIs handle the creation and management of timetables. This includes managing timetable entries, viewing schedules, and exporting timetables as PDF documents. The system supports real-time timetable generation with features such as room allocation and break or lunch scheduling.

The Notification APIs are responsible for managing communication features within the system. These include real-time notifications, an email alert system, and notification history tracking. This ensures that users are instantly informed about updates and deadlines.

3. ML / Analytics Layer

The ML and Analytics Layer enhances the system by introducing intelligence and automation through machine learning algorithms. This layer significantly improves the overall user experience.

The LightGBM model is used for notice ranking by assigning priority scores based on factors such as urgency, relevance, and user engagement. This ensures that important notices are displayed at the top. The DistilBERT model is used for smart search functionality by understanding the semantic meaning of notice content, thereby enabling accurate and efficient search results.

The Cox Proportional Hazards Model is used for risk prediction by estimating the probability of users missing deadlines and triggering alerts for important notices. This helps students manage their time effectively. The Singular Value Decomposition (SVD) technique is used for building a recommendation system that suggests notices based on user behavior such as clicks, views, and downloads. This provides a personalized user

experience. Overall, this layer transforms the system from a basic notice board into a smart, adaptive, and predictive platform.

4. Data and Storage Layer

The Data and Storage Layer plays a crucial role in managing and storing all application data in a structured, secure, and efficient manner. It ensures that data is consistently maintained, easily retrievable, and protected from unauthorized access. The system uses a relational database to store structured data such as user details, department information, notices with their content and approval status, timetable data including schedules and room allocation, and notification records. This ensures data consistency, integrity, and efficient querying.

In addition to structured data, the system also supports file storage for managing uploaded files such as PDF documents, images, and other formats including audio, video, Excel, and presentations. This ensures secure storage, easy access, and proper organization of files. The system also maintains interaction logs to track user behavior, including notice views, clicks, and downloads. This data is used for generating analytics, training machine learning models, and providing personalized recommendations.

5. System Communication Flows

The system follows a structured communication flow to ensure efficient operation. Users interact with the system through the React-based user interface, and their requests are sent via HTTPS-based REST APIs to the Django backend. The Application Layer processes these requests and interacts with both the database for data storage and retrieval, and the ML layer for ranking, prediction, and recommendation tasks. The processed results are then returned to the frontend for display. Additionally, notifications are triggered whenever necessary to keep users informed in real time.

3.4 Methodology

The methodology of the proposed Digital Notice Board and Alert Management System is designed to provide an intelligent, efficient, and user-centric approach to managing academic notices within an educational institution. The system integrates modern web technologies with machine learning algorithms to ensure that users receive relevant, timely, and prioritized information. The overall working process follows a structured pipeline that begins with user authentication and ends with personalized notifications and alerts. Each stage of the methodology is carefully designed to ensure accuracy, scalability, and improved user experience.

Furthermore, the system emphasizes modular design, allowing different components such as authentication, notice management, and notification services to operate independently while maintaining seamless integration. Data-driven decision-making is incorporated through continuous analysis of user interactions, enabling the system to adapt and improve over time. The methodology also ensures efficient handling of large volumes of data,

making the system suitable for institutions with a high number of users. Additionally, the use of secure communication protocols and role-based access control enhances system reliability and protects sensitive information.

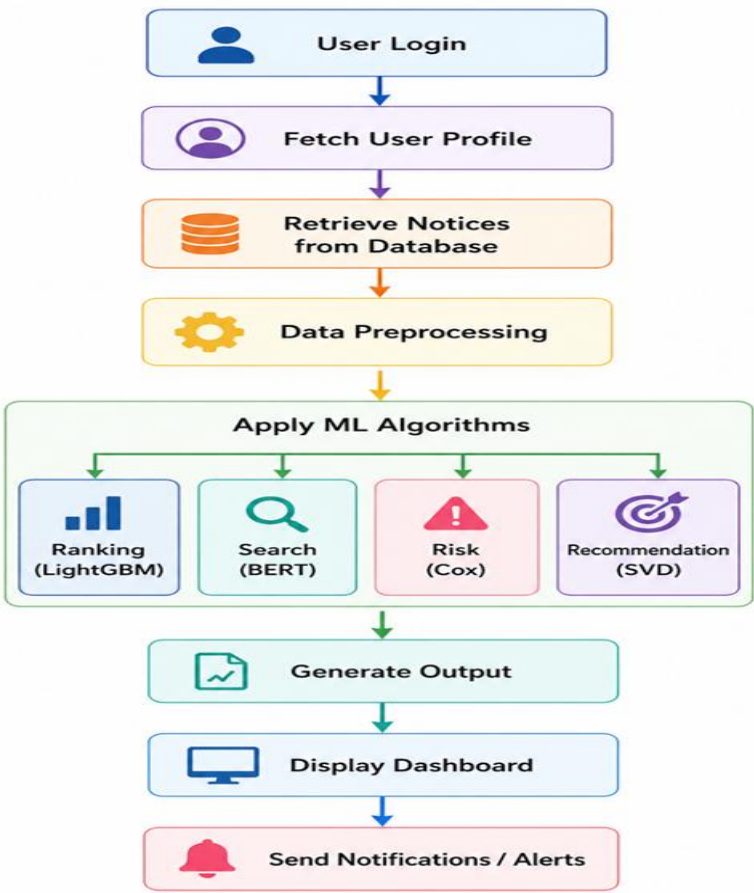


Figure 2: Methodology

3.4.1 User Authentication and Login Process

The first step in the system workflow is user authentication, where users log into the system using their credentials. The system supports multiple roles including Student, Staff, Department, HOD, and Admin, each having specific permissions and access rights. Authentication is implemented using secure mechanisms such as JWT (JSON Web Token) or session-based authentication to ensure data security and prevent unauthorized access.

3.4.2 User Profile Retrieval

After successful login, the system retrieves the user profile information from the database. This includes details such as user role, department, academic year, preferences, and interaction history. The profile plays a critical role in personalizing the system output.

For example, a student belonging to the Computer Science department will receive notices related to their department, while staff members will have access to departmental

management features. The system uses this profile data as a foundation for further processing, including filtering notices and generating personalized recommendations.

3.4.3 Retrieval of Notices from Database

In this step, the system retrieves all relevant notices from the database to ensure that users have access to updated and categorized information. The notices include various categories such as academic notices, examination schedules, placement updates, workshop announcements, scholarships, and events. The database stores each notice along with important metadata, including the title, description, category, department, deadline, and any associated attachments. The system fetches notices dynamically based on the user's role and access permissions. For instance, students are allowed to view only published notices, whereas Heads of Department (HODs) can access pending notices that require approval. This step ensures that the system has access to complete and relevant data required for further processing, analysis, and intelligent decision-making.

3.4.4 Data Preprocessing

Before applying machine learning algorithms, the retrieved data undergoes a preprocessing stage to enhance its quality and usability. This step involves cleaning the text data by removing unnecessary characters and symbols, normalizing text to maintain consistency, and extracting important keywords from notice content.

Additionally, the system handles missing or incomplete data and organizes the information into a structured format suitable for machine learning models. For example, notice descriptions are processed to remove stop words and converted into a structured representation that can be easily interpreted by Natural Language Processing (NLP) models.

3.4.5 Application of Machine Learning Algorithms

After preprocessing, the system applies multiple machine learning algorithms to enhance functionality and intelligence. These algorithms collectively provide capabilities such as ranking, smart search, risk prediction, and personalized recommendation.

The LightGBM algorithm is used for notice ranking by assigning a priority score to each notice. It considers factors such as notice importance, proximity of deadlines, user engagement in terms of views and clicks, and category relevance. Based on these factors, notices are ranked so that high-priority items appear at the top of the dashboard, enabling users to quickly identify critical information.

The DistilBERT model, a lightweight Natural Language Processing model, is used for smart search functionality. Unlike traditional keyword-based search methods, this model understands the semantic meaning and context of the text. For instance, when a user searches for “exam schedule,” the system can retrieve related notices even if the exact keywords are not present, thereby improving search accuracy and user experience.

The Cox Proportional Hazards Model is used for risk prediction to estimate the likelihood of users missing important deadlines. It analyzes factors such as the remaining time until the deadline, user engagement level, and notice importance. Based on the computed risk score, the system generates alerts when the probability of missing a deadline is high, helping users manage their time effectively.

The Singular Value Decomposition (SVD) technique is used to build a recommendation system that provides personalized notice suggestions. It analyzes user behavior, including viewed notices, clicked items, and downloaded attachments. Based on this interaction data, the system recommends relevant notices tailored to each user's interests and needs, thereby enhancing user engagement and experience.

3.4.6 Output Generation

After the application of machine learning models, the system generates the final output in a structured and user-friendly format. The output includes a ranked list of notices based on priority, filtered results tailored to the user's profile, personalized recommendations, and risk alerts for approaching deadlines. The information is presented in a visually organized manner, where notices are grouped by categories and displayed in order of importance. This ensures that users can easily understand, navigate, and act upon the information provided by the system.

3.5 Algorithms

The proposed Digital Notice Board and Alert Management System uses four intelligent algorithms to make the system smart, personalized, and predictive. These algorithms help in ranking notices, understanding text, predicting deadline risks, and recommending relevant content to users. Each algorithm plays a specific role in improving the overall efficiency and user experience of the system.

3.5.1. LightGBM – Notice Priority Ranking

The LightGBM algorithm is used to rank notices based on their importance so that students can easily identify the most relevant and urgent information. In a traditional system, all notices are displayed equally, which may cause important notices to be overlooked. However, in this system, LightGBM assigns a priority score to each notice based on various factors such as department relevance, student year, notice category, importance level, and the number of days remaining before the deadline.

LightGBM is a tree-based machine learning algorithm that works by combining multiple decision trees to produce accurate predictions. It is chosen because it is fast, efficient, and suitable for structured data. The algorithm evaluates multiple conditions and calculates a score that represents the importance of each notice.

Mathematical Idea (Simple Form)

Prediction is calculated as the sum of outputs from multiple trees:

$$Prediction = \sum_{i=1}^n Tree_i(x)$$

Here, each tree contributes a small value, and the final score is obtained by combining all tree outputs.

Working Process: First, the system collects notice data and student details. Then, the features are converted into numerical format. The model is trained using historical data. For every new notice, the system calculates a feature vector and predicts a priority score between 0 and 1. Finally, notices are sorted based on this score and displayed in ranked order.

LightGBM – Notice Priority Ranking

Input: Notice set N, Student profile S (department, year, preferences)

Output: Ranked notice list N_ranked with priority scores

```
// 1. Feature Extraction
1. for each notice n_i in N do
    x_i ← EXTRACT_FEATURES(n_i, S)
    // features: dept_relevance, student_year, category, importance, days_left
end for
// 2. Priority Score Prediction
2. for each feature vector x_i do
    score_i ←  $\sum_{k=1}^K Tree_k(x_i)$  // sum of K tree outputs
end for
// 3. Normalisation & Ranking
3. score_i ← SIGMOID(score_i) // map to [0, 1]
4. N_ranked ← SORT_DESC(({n_i, score_i}))
return N_ranked
```

3.5.2 DistilBERT – Smart Notice Search and Understanding

DistilBERT is used to improve the search functionality of the system by understanding the meaning of notice content rather than just matching keywords. In traditional systems, search results are based only on exact word matches, which may not always provide accurate results. DistilBERT solves this problem by using Natural Language Processing (NLP) to understand the context of the text.

This algorithm converts notice text into a numerical representation called a vector (embedding). It is lightweight and faster compared to other NLP models, making it suitable for web applications.

Mathematical Idea (Simple Flow)

Text $\rightarrow$ *Tokenization* $\rightarrow$ *Embeddings* $\rightarrow$ *Transformer* $\rightarrow$ *ContextVector*

This means that the input text is broken into tokens, converted into embeddings, processed through transformer layers, and finally converted into a contextual vector.

Working Process: The system takes the notice title and description as input and tokenizes the text. Each word is converted into embeddings and passed through the model to generate a contextual vector. These vectors are stored in the database. When a user searches, the query is also converted into a vector, and similarity is calculated to find the most relevant notices.

DistilBERT – Smart Notice Search and Understanding

Input: Query string q , Notice corpus C

Output: Ranked relevant notice list R

```
// 1. Corpus Embedding (offline / pre-computed)
1. for each notice  $n_i$  in  $C$  do
    tokens_i  $\leftarrow$  TOKENIZE( $n_i$ .title ||  $n_i$ .description)
    embed_i  $\leftarrow$  DISTILBERT(tokens_i) // contextual vector
    STORE(embed_i,  $n_i$ .id)
end for
// 2. Query Embedding
2. tokens_q  $\leftarrow$  TOKENIZE( $q$ )
3. embed_q  $\leftarrow$  DISTILBERT(tokens_q)
// 3. Similarity Search
4. for each stored embed_i do
    sim_i  $\leftarrow$  COSINE_SIM(embed_q, embed_i)
    = (embed_q  $\cdot$  embed_i) / ( $\|$ embed_q $\|$   $\cdot$   $\|$ embed_i $\|$ )
end for
// 4. Result Ranking
5.  $R \leftarrow$  SORT_DESC( $\{(n_i, sim_i)\}$ )

return  $R$ 
```

3.5.3. Cox Proportional Hazard Model – Deadline Risk Prediction

The Cox Proportional Hazard Model is used to predict the probability of a student missing a deadline. This is important because students often forget or overlook deadlines. This algorithm analyzes time-based data and calculates a risk score.

The model considers factors such as the number of days remaining, notice importance, and user engagement (views or clicks). Based on these inputs, it determines whether the student is likely to miss the deadline.

Mathematical Formula

$$h(t) = h_0(t) \times e^{(\beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n)}$$

Where:

- $h(t)$ = Risk of missing deadline
- $h_0(t)$ = Base risk
- x = Features (days left, engagement, importance)
- β = Coefficients

Working Process: The system calculates the remaining days for each notice and checks user interaction history. These values are passed into the Cox model to compute a risk probability. If the risk exceeds a certain threshold, the system automatically sends reminders or alerts to the user.

Cox Proportional Hazard Model – Deadline Risk Prediction

Input: Notice n_i , User interaction history H_u , Current time t_{now}

Output: Risk score $\rho \in [0, 1]$, Alert decision $\alpha \in \{\text{ALERT}, \text{SILENT}\}$

```
// 1. Feature Construction
1. days_left ← n_i.deadline – t_now
2. importance ← n_i.importance_level
3. engagement ← COUNT_INTERACTIONS(H_u, n_i.id) // views + clicks
4. x ← [days_left, importance, engagement]
// 2. Hazard Computation
5. h(t) ← h_0(t) × exp(β_1·days_left + β_2·importance + β_3·engagement)
   // h_0(t) = baseline hazard; β = learned coefficients
// 3. Risk Score Derivation
6. ρ ← 1 – exp(–∫_0^t h(s) ds) // cumulative risk
// 4. Alert Decision
7. θ ← LOAD_THRESHOLD() // e.g. 0.70
8. if ρ ≥ θ then
   SEND_REMINDER(u, n_i)
   α ← ALERT
else
   α ← SILENT
end if
return (ρ, α)
```

3.5.4. Matrix Factorization – Personalized Recommendation

Matrix Factorization is used to provide personalized recommendations to users based on their interests and past interactions. In a traditional notice system, all users see the same notices. However, this system recommends notices that are most relevant to each user.

The algorithm works by analyzing user behavior such as views, clicks, and downloads. It creates a relationship between users and notices and predicts which notices a user is likely to be interested in.

Mathematical Idea

$$R \approx P \times Q^T$$

Where:

- R = User-Notice interaction matrix
- P = User preference matrix
- Q = Notice feature matrix

Working Process: The system first builds an interaction matrix using user activity data. Then, matrix factorization is applied to break this matrix into smaller matrices representing user preferences and notice features. Using these matrices, the system predicts unseen interactions and recommends relevant notices to users.

Matrix Factorization – Personalized Recommendation

Input: Interaction matrix R (users $\times$ notices), User u , Top-K value K

Output: Personalised recommendation list Rec_u

// 1. Matrix Decomposition

1. $(P, Q) \leftarrow \text{SVD_FACTORIZE}(R)$

// $R \approx P \times Q^T$

// $P \in \mathbb{R}^{\{|U| \times d\}}$: user latent factors

// $Q \in \mathbb{R}^{\{|N| \times d\}}$: notice latent factors

// 2. Score Prediction for Unseen Notices

2. $U_unseen \leftarrow \{n_j \in N : R[u, n_j] = 0\}$ // not yet interacted

3. for each n_j in U_unseen do

$\hat{r}_{\{u,j\}} \leftarrow P_u \cdot Q_j^T$ // dot product

end for

// 3. Top-K Selection

4. $Rec_u \leftarrow \text{TOP_K}(\{(n_j, \hat{r}_{\{u,j\}})\}, K)$

// 4. Optional Re-ranking with Priority Scores

5. $Rec_u \leftarrow \text{MERGE_RANK}(Rec_u, N_ranked)$ // blend with LightGBM scores

return Rec_u

3.6 Data Flow Diagram



Figure 3 : Data flow Diagram

Level 0 (Context Diagram)

The Level 0 Data Flow Diagram represents the overall interaction between the system and external entities. The Academic Notice and Timetable System acts as the central system that communicates with five major external users: Student, Staff, HOD (Head of Department), Department Admin, and System Admin. These users provide inputs such as login credentials, notice requests, approvals, and timetable details. In response, the system processes these inputs and generates outputs including notices, approval results, notifications, schedules, and reports. This level provides a high-level overview of how data flows between users and the system without showing internal processes.

Level 1 (Major Processes)

The Level 1 Data Flow Diagram breaks down the system into major functional processes, each handling specific operations:

1. Authentication Process

This process verifies user identity. Users provide login credentials as input, and the system validates them using the user database. Upon successful verification, the system generates an authentication token or session along with the assigned user role.

2.Notice Management Process

In this process, staff members create and submit notices. These notices are stored in the notice database and sent for approval. Once approved, they are published and made available to students. Attachments related to notices are stored separately in file storage.

3.Approval Process

The HOD or admin reviews submitted notices and either approves or rejects them. Based on their decision, the system updates the notice status. All approval actions are recorded in the approval records database for tracking and auditing.

4.Ranking and Recommendation Process

This process enhances user experience by analyzing user interactions and notice-related data. Based on this input, the system generates a ranked feed of notices and personalized recommendations. The data is stored in interaction logs and model output storage.

5.Timetable Generation Process

This module generates academic timetables using inputs such as subjects, staff availability, classrooms, and time slots. The output includes structured timetable entries and downloadable PDF reports. The data is stored in the timetable database and report storage.

6.Notification Process

This process ensures real-time communication. Whenever a new notice is published or a status changes, the system generates alerts and sends notifications to relevant users. All notifications are logged in the notification database for reference.

3.7 Module Description

1. Authentication Module

This module controls access to the system and ensures that only authorized users can access the platform. It manages user registration for students and department staff, verifies login credentials using JWT tokens, and assigns role-based permissions (Student, Staff, Department, HOD, Admin). The module enforces secure password handling, manages session timeouts, handles logout functionality, and performs account status verification. Security measures include input validation, SQL injection prevention, and role-based access control (RBAC) to ensure users can only access features permitted by their role.

2. Student Dashboard Module

The student dashboard serves as the primary interface for students to access notices and announcements. It displays notices in a personalized feed ranked by relevance, deadline urgency, and category. Students can search notices using semantic search, filter by department or category, view attachment details, and track read/unread status. The dashboard highlights deadline-risk notices with visual indicators and provides quick access

to timetables and reminders. ML algorithms rank notices to show the most relevant content first, reducing information overload.

3. Staff Dashboard Module

The staff dashboard provides for Staff members can draft notices, attach files, assign categories, set priority levels, and schedule publication. Staff can edit or withdraw notices before final approval and receive notifications when HODs or admins take action. This transparency keeps staff informed throughout the notice lifecycle.

4. Department Module

This module manages department-level operations and academics. Department users can submit notices, view departmental analytics, and access timetable management features. The module supports department profile management, staff assignment, course enrollment, and notification preferences. Departments can generate conflict-free timetables by inputting subjects, staff, rooms, and time slots. Features include timetable filtering by staff, room, or day, and PDF export for easy distribution to students and staff.

5. HOD (Head of Department) Module

The HOD module implements the first-level approval workflow for notices. HODs review notices submitted by staff in their department, have the authority to approve or reject notices, and can add remarks or request corrections.

6.Admin Dashboard Module

The admin dashboard provides system-wide oversight and control. Admins can approve or reject notices after HOD review, manage user accounts (create, deactivate, reset passwords), manage departments, and monitor system activity logs. The module includes analytics dashboards showing notice performance metrics, user engagement trends, system health indicators, and departmental communication effectiveness.

7.Notification and Alert Module

This module delivers real-time notifications across the platform using WebSocket technology. It notifies users of new notices, approval status changes, deadline reminders, and timetable updates. The module tracks notification status (read, unread, acknowledged), enabling follow-up actions and engagement analytics. Priority-based alert routing ensures urgent notices reach users immediately.

8.Timetable Generation Module

This module automates academic schedule creation and conflict resolution. It accepts structured inputs (subjects, staff, room availability, time slots, course schedules) and applies scheduling algorithms to prevent overlaps and conflicts. Generated timetables are stored in the database with version control and can be filtered by staff, room, or time slot.

Features include visual timetable display, PDF export for printing and distribution, and update history tracking.

9. Notice Ranking and Recommendation Module

This ML-driven module personalizes notice feeds for each user. It uses features such as department affiliation, category preferences, deadline proximity, and historical engagement patterns to rank notices by relevance. The module continuously learns from user interactions (views, clicks, time spent) to improve recommendations. Students see the most relevant notices at the top, while older or less relevant notices move lower, reducing information overload.

10. Analytics and Module

This module tracks user interactions with notices and system features. It records metrics such as notice views, click-through rates, time spent on content, and completion of actions. Analytics help departments optimize their notice distribution strategy and measure institutional communication impact

CHAPTER 4

IMPLEMENTATION AND RESULTS

4.1 Tools & Technologies

4.1.1 Hardware Requirements

The application is designed to be lightweight and accessible, requiring only standard consumer-grade hardware. It can run on a basic Intel Core i3 processor with a minimum of 4 GB RAM and 500 MB of free storage, making it deployable on most institutional or personal computers without the need for dedicated servers. A stable broadband internet connection is necessary to support email delivery, GitHub integration. For a smoother development and testing experience, an Intel Core i5 with 8 GB or more of RAM is recommended.

Table 2: Hardware Requirements

Component	Minimum Specification	Recommended Specification
Processor	Intel Core i3	Intel Core i5
RAM	4 GB	8 GB or more
Storage	500 MB free	1 GB free
Network	Broadband internet	Broadband internet

4.1.2 Software Requirements

The project is built entirely on open-source technologies, requiring no paid licenses. The frontend is developed using HTML5, CSS3, and JavaScript (ES6) with Vite 5.x as the build and development server. The backend is powered by Python 3.11 with Flask handling API routing and request management. SQLite 3 serves as the embedded relational database, eliminating the need for a separate database server. The entire project is developed on Windows 11 using Visual Studio Code as the primary IDE, tested on Google Chrome, and version-controlled through Git and GitHub.

Table 3: Software Requirements

Component	Technology / Version
Operating System	Windows 11
Frontend	HTML5, CSS3, JavaScript (ES6), Vite 5.x
Backend	Python 3.11, Flask / Built-in HTTP Server
Database	SQLite 3
E-mail	SMTP (smtplib, email.mime)
IDE	Visual Studio Code
Browser	Google Chrome
Version Control	Git and GitHub

4.2 System Implementation

The implementation of the Digital Notice Board (DNB) system follows a three-tier architectural model, which separates the application into distinct layers for better maintainability and scalability. The frontend layer is responsible for presenting the user interface and handling user interactions across role-based dashboards. The backend layer processes user requests, applies business logic, handles ML-based operations, and communicates with the database. The database layer stores all application data including user details, notices, alerts, timetables, and department configurations. This separation of concerns ensures that each component can be independently developed and maintained, resulting in a robust and efficient system.

File Structure

```
DNB/
|
|— .venv/                # Python virtual environment
|— backend/
|   |— apps/
|   |   |— timetable/    # Timetable generation module
|   |   |   |— views.py  # API route handlers
|   |   |   |— algorithm.py # Timetable scheduling algorithm
|   |   |   |— models.py  # Data models
|   |   |— notices/      # Notice management module
|   |   |— alerts/       # Alert management module
|   |   |— users/        # User & role management
|   |— config.py         # Configuration / env vars
|   |— db.py             # SQLite access layer
|   |— mailer.py         # Automated e-mail engine
|   |— app.py            # App bootstrap / entry point
|   |— dnb.db            # SQLite database file
|— database/
|   |— dnb_schema.sql    # DDL script
|— email_logs/
```

```

|   └─ email_log.txt
└─ frontend/
    |   └─ src/
    |   |   └─ features/
    |   |   |   └─ department/
    |   |   |   |   └─ pages/
    |   |   |   |   |   └─ TimetableGenerator.jsx
    |   |   |   └─ notices/
    |   |   └─ dashboard/
    |   └─ components/      # Shared UI components
    |   └─ main.jsx         # React entry point
    └─ index.html
    └─ package.json
    └─ package-lock.json
    └─ vite.config.js
└─ extract_pdf.py          # PDF extraction utility
└─ generate_doc.py         # Document generation script
└─ .env
└─ .env.example
└─ README.md

```

4.2.1 Frontend Implementation

The frontend of the DNB system is designed to provide a user-friendly and interactive interface that allows users to navigate the platform with ease based on their assigned roles. It is developed using HTML5 for structure, CSS3 for styling, and JavaScript (ES6) for dynamic behavior, with React as the component framework and Vite 5.x as the build tool for fast module handling and hot-module replacement during development.

The application includes several key pages that support different functionalities. The dashboard serves as the role-aware entry point, adapting its content based on whether the user is an administrator, staff member, or viewer. The notice board page displays active notices filtered by department and priority. The timetable generator allows department heads to configure and auto-generate conflict-free schedules.

The alert management page enables real-time notifications to be broadcast to specific user groups. The admin panel provides full control over users, departments, and system configurations.

The frontend of the system is structured using multiple components and pages to ensure a smooth and user-friendly experience. The `index.html` file acts as the application shell, initializing and bootstrapping the React Single Page Application (SPA). The Timetable Generator component provides a multi-step interface for configuring and generating timetables, along with a live preview to help users visualize schedules before finalizing them. The Dashboard page serves as the main interface, displaying a role-based overview of notices, alerts, and upcoming schedules for users.

The Notice Board page is responsible for displaying all active notices with additional details such as priority badges, department tags, and expiry indicators, making it easier for users to identify important updates. The Alert Management page allows administrators and departments to create, broadcast, and monitor real-time alerts across the system. The User Management page enables administrators to handle user registration, assign roles, and control access permissions securely. Finally, the Profile page provides users with options to manage their personal information, notification preferences, and account settings, ensuring a personalized and controlled user experience.

The frontend interacts with the backend through RESTful API calls, ensuring real-time data exchange. Client-side form validation is implemented to ensure accurate and complete information before submission. Overall, the frontend is designed to enhance usability and provide a seamless, role-driven user experience.

4.2.2 Backend Implementation

The backend of the DNB system is developed using Python 3.11 with Flask handling the core business logic and API routing. It acts as the central processing unit connecting the frontend interface with the database layer. The backend is responsible for user authentication, notice and alert management, timetable scheduling via a constraint-based algorithm, PDF extraction, and automated email notifications.

When a notice is published by an authorized staff member, the backend validates the input, stores it in the database, and triggers notifications to relevant user groups. The timetable generation module applies a scheduling algorithm that resolves conflicts across subjects, teachers, and rooms before returning a finalized timetable to the frontend. The backend enforces role-based access control ensuring each user can only perform actions permitted by their assigned role.

Key backend modules:

Table 4: Key backend modules

Module	Responsibility
app.py	Application bootstrap and Flask initialization
apps/timetable/views.py	API endpoints for timetable operations
apps/timetable/algorithm.py	Constraint-based scheduling algorithm
apps/notices/views.py	Notice CRUD and publishing logic
apps/alerts/views.py	Alert broadcasting and tracking
mailer.py	Automated SMTP email engine
db.py	SQLite query interface and connection management
config.py	Environment variable management

4.2.3 Database Implementation

The system uses SQLite 3 as the database management system due to its simplicity and efficiency for small to medium-scale institutional applications. The database is designed to store all data required for the operation of the system with a clean relational schema.

The users table stores user ID, name, email, hashed password, role, and department. The notices table records notice content, department target, priority level, publish date, and expiry date. The alerts table stores alert messages, severity level, broadcast scope, and acknowledgement status. The timetable table holds generated schedules linked to departments, subjects, teachers, and room assignments. The email_logs table tracks all outbound email events including recipient, type, timestamp, and delivery status.

4.2.4 Database Design

The database uses a relational model to ensure integrity and reduce redundancy. The main design principles are:

1. User table

The User table is the central entity of the database. It stores the essential details of every person registered in the system, including a unique user identifier, full name, email address, role (Student, Staff, HOD, or Admin), encrypted password, and account status (Active or Inactive). This table maintains a one-to-many relationship with notices, approvals, and notifications.

Table 5 : User table

Column Name	Data Type	Description
user_id	INTEGER (PK)	Unique user ID
name	TEXT	User full name
email	TEXT (UNIQUE)	User email
role	TEXT	Student / Staff / HOD / Admin
password_hash	TEXT	Encrypted password
status	TEXT	Active / Inactive

2.Student Profile table

The Student Profile table extends the User table by storing academic-specific information. Each profile is linked to a user and a department through foreign keys. It additionally records the student's academic year and the staff designation, such as Professor or HOD. Each profile belongs to exactly one user and one department.

Table 6 : StudentProfile table

Column Name	Data Type	Description
profile_id	INTEGER (PK)	Unique profile ID
user_id	INTEGER (FK)	Reference to USERS
department_id	INTEGER (FK)	Reference to DEPARTMENT
year	INTEGER	Student academic year
designation	TEXT	Staff role (Professor / HOD)

3.Department table

The Department table holds the details of each academic department in the institution. It stores a unique department identifier, the department name, and a short code such as CSE or ECE. This table has a one-to-many relationship with both staff members and notices. It plays a crucial role in organizing and categorizing notices based on departments, ensuring that users receive relevant information. Additionally, it helps in maintaining structured data management and supports efficient filtering and retrieval of department-specific records.

Table 7: Department table

Column Name	Data Type	Description
department_id	INTEGER (PK)	Unique department ID
name	TEXT	Department name
code	TEXT	Department code (CSE, ECE)

4. Notice table

The Notice table is the core content table of the system. Each notice record contains a unique identifier, title, body text, category (such as Exam, Event, or Placement), the identifier of the user who created it, the associated department, creation and expiry timestamps, and the approval status from both the HOD and the Admin. A notice can have many attachments and can be linked to many user notifications.

Table 8: Notice table

Column Name	Data Type	Description
notice_id	INTEGER (PK)	Unique notice ID
title	TEXT	Notice title
body	TEXT	Notice description
category	TEXT	Exam / Event / Placement
created_by	INTEGER (FK)	Reference to USERS
department_id	INTEGER (FK)	Reference to DEPARTMENT
created_at	DATETIME	Creation timestamp
expiry_at	DATETIME	Expiry date
hod_status	TEXT	Approved / Pending
admin_status	TEXT	Approved / Pending

5. Notice Attachment table

The Notice Attachment table manages files associated with a notice, such as PDFs or images. Each attachment record stores a unique identifier, a reference to the related notice, the file storage path, and the upload timestamp. Multiple attachments can belong to a single notice.

Table 9:Notice Attachment table

Column Name	Data Type	Description
attachment_id	INTEGER (PK)	Unique attachment ID
notice_id	INTEGER (FK)	Reference to NOTICE
file_path	TEXT	File path (PDF/Image/etc.)
uploaded_at	DATETIME	Upload timestamp

6.User Notification table

The User Notification table links notices to individual users for targeted delivery. It records a unique notification identifier, the user reference, the notice reference, a read or unread status flag, and the timestamp when the notification was created. This table ensures that each notice is properly delivered and tracked at the user level.

Table 10:User Notification table

Column Name	Data Type	Description
notification_id	INTEGER (PK)	Unique notification ID
user_id	INTEGER (FK)	Reference to USERS
notice_id	INTEGER (FK)	Reference to NOTICE
is_read	BOOLEAN	Read / Unread
created_at	DATETIME	Notification time

7.Timetable Batch table

The TimetableBatch table stores metadata for each generated timetable. It includes a unique batch identifier, the associated department, the user who created the batch, and the creation timestamp. Each batch can contain multiple timetable entries. This table acts as a parent record that groups all individual timetable slots under a single generation event, making it easy to manage and retrieve complete timetables by department. It also supports version tracking, allowing the system to maintain multiple timetable generations for the same department across different time periods.

Table 11 :Timetable Batch table

Column Name	Data Type	Description
batch_id	INTEGER (PK)	Unique batch ID
department_id	INTEGER (FK)	Reference to DEPARTMENT
created_by	INTEGER (FK)	Reference to USERS
created_at	DATETIME	Created time

8.Timetable Entry table

The Timetable Entry table stores the individual slots of a generated timetable. Each entry is linked to a batch and records the day, time slot, subject name, assigned staff member, and classroom. Multiple entries belong to a single timetable batch.

Table 12:TimeTable Entry table

Column Name	Data Type	Description
entry_id	INTEGER (PK)	Unique entry ID
batch_id	INTEGER (FK)	Reference TIMETABLE_BATCH
day	TEXT	Monday / Tuesday
time_slot	TEXT	9-10 / 10-11
subject	TEXT	Subject name
staff_id	INTEGER (FK)	Staff reference
room	TEXT	Classroom

9.Interaction table

The Interaction table serves as a behavioural log for the machine learning components of the system. It records every user interaction with a notice, including a unique log identifier, the user reference, the notice reference, the type of action performed (View, Click, or Download), and the timestamp of the interaction. This data is used by the LightGBM ranking engine and the Matrix Factorization recommendation system to deliver personalised and prioritised notice experiences. This gives you clean, flowing paragraph text suitable for a thesis chapter without any tables or bullet points.

Table 13: Interaction table

Column Name	Data Type	Description
log_id	INTEGER (PK)	Unique log ID
user_id	INTEGER (FK)	Reference to USERS
notice_id	INTEGER (FK)	Reference to NOTICE
action_type	TEXT	View / Click / Download
timestamp	DATETIME	Interaction time

4.2.5 Email Automation System

An important feature of the DNB system is its automated email communication system, which enhances user experience by delivering timely notifications without manual intervention. The email automation is implemented using Python's Smtplib And email .mime libraries, triggered at key system events.

When a high-priority alert is broadcast, the system automatically sends notification emails to all users within the target department. When a new notice is published, email summaries are dispatched to subscribed users. Timetable generation completion events also trigger email delivery to relevant staff with the finalized schedule attached. All outbound emails are logged in the email_logs table for traceability and audit purposes.

4.2.6 Timetable Generation

A core feature of the DNB system is its automated timetable scheduling engine. This module enables department administrators to configure constraints such as available teachers, subjects, classrooms, and time slots, and automatically generate a conflict-free timetable through a constraint-satisfaction algorithm.

When a department head submits a configuration, the frontend sends the parameters to the backend scheduling algorithm. The algorithm iterates through time slots and assigns subjects, teachers, and rooms while checking for conflicts. Once a valid schedule is produced, it is stored in the database and returned to the frontend for display and export. This significantly reduces the manual effort involved in academic scheduling and eliminates human errors.

4.3 Working Model

The student workflow within the Digital Notice Board system is designed to provide a seamless and informative experience. Once a student logs in, the system identifies their role and department, and directs them to a personalized dashboard showing only the content relevant to them. Students can view notices, check their timetable, receive alerts, and stay

updated on department activities all without any administrative privileges. The workflow ensures students remain informed in real time without requiring manual inquiry.

STEP 1: Student Login Page The student enters their registered email and password. The system authenticates the credentials, resolves the student role, and establishes a secure session.

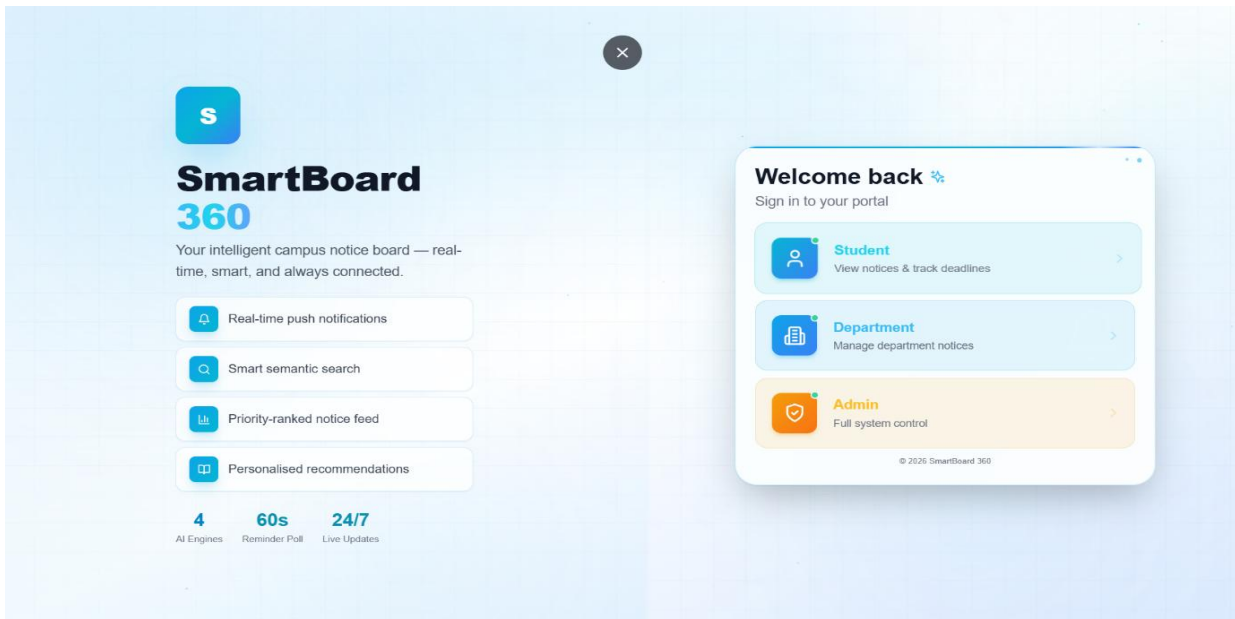


Figure 4: login page

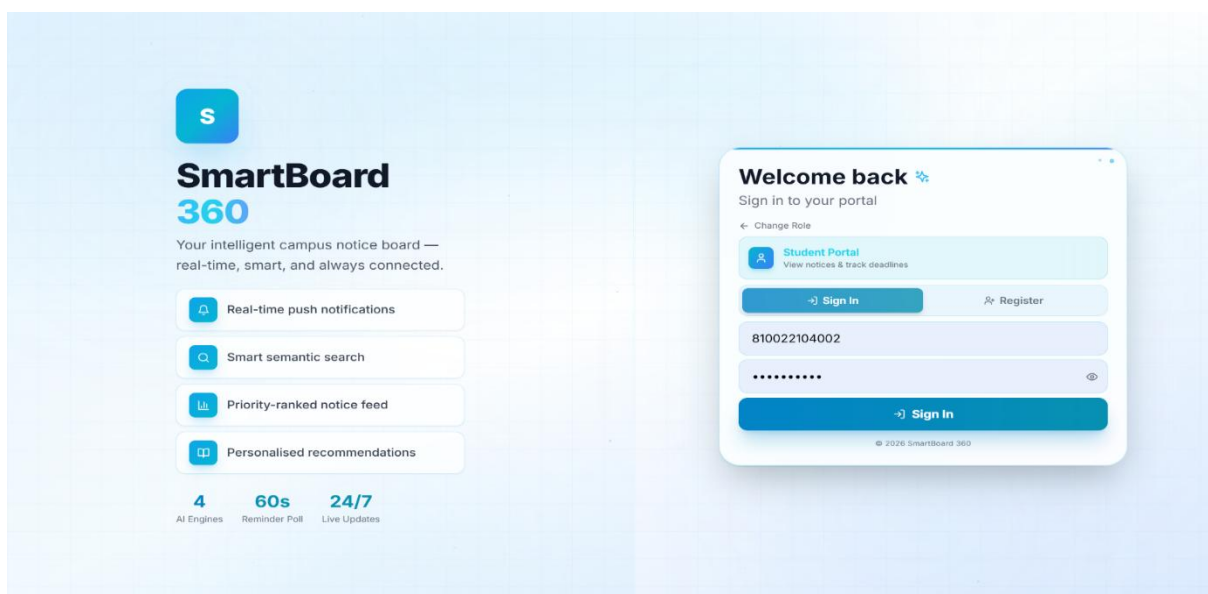


Figure 5: student login

STEP 2: Student Dashboard Up on login, the student is directed to their role-specific dashboard. The dashboard displays a summary of recent notices, active alerts, and their current timetable — filtered automatically by their enrolled department.

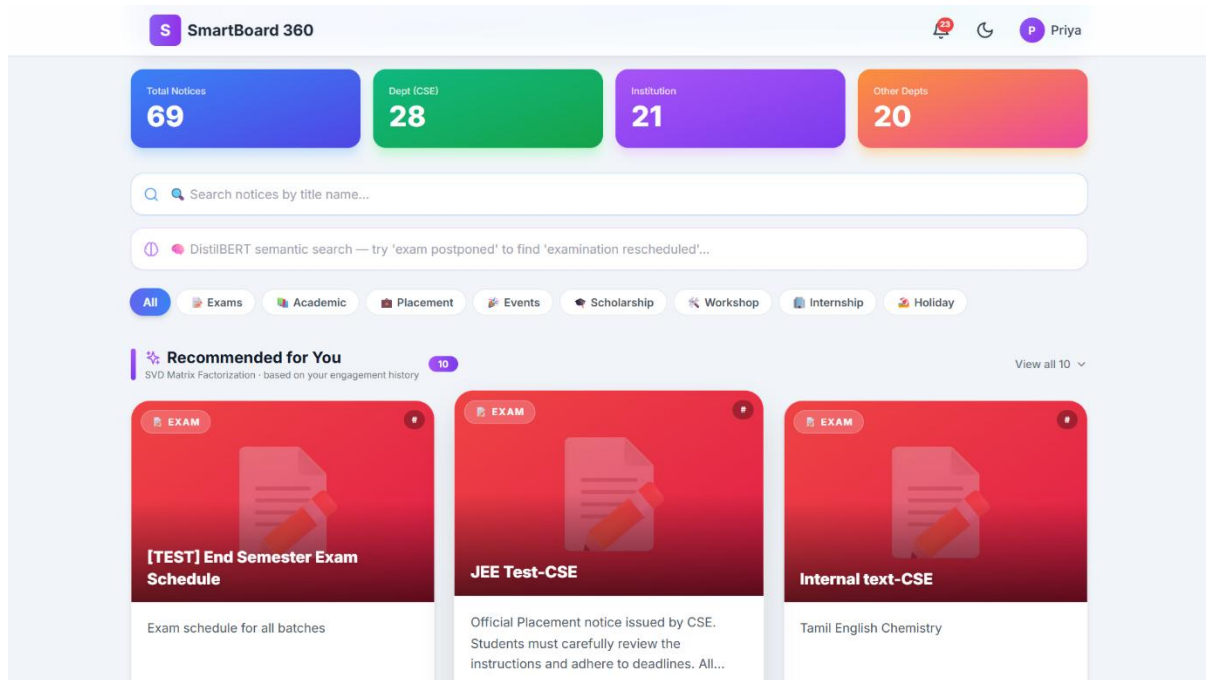


Figure 6: Student Dashboard

STEP 3: Viewing the Notice Board The student navigates to the Notice Board page. Notices are displayed in order of priority. Each notice shows the title, department tag, publish date, and expiry date.

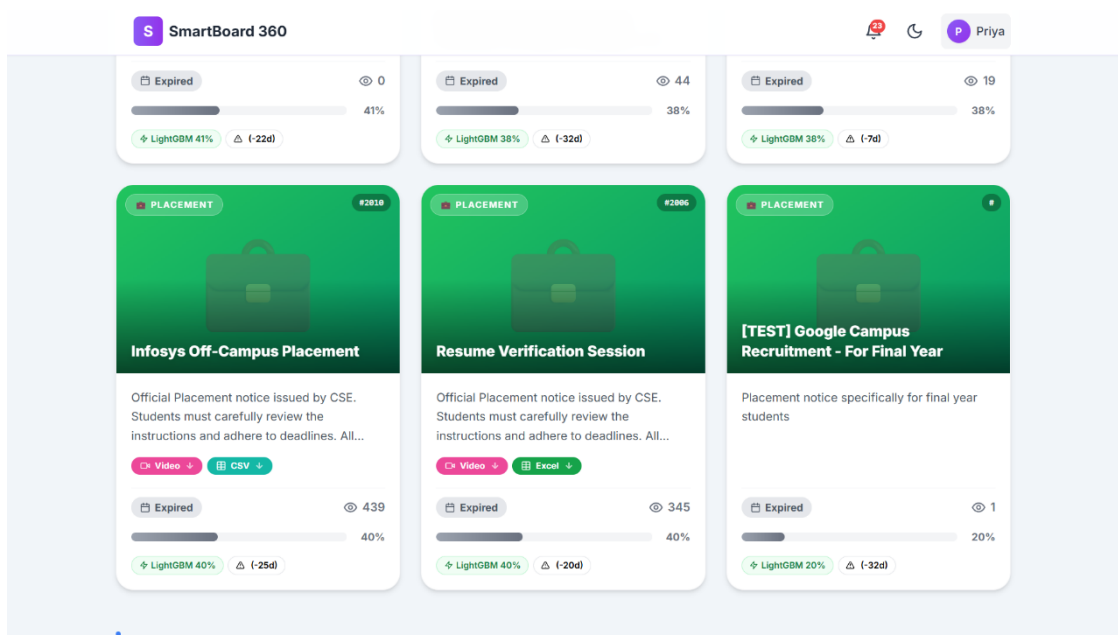


Figure 7: Student Notice

STEP 4: Reading a Notice The student clicks on a notice to view its full content. The detailed view shows the complete message, the targeted department, and the validity period.

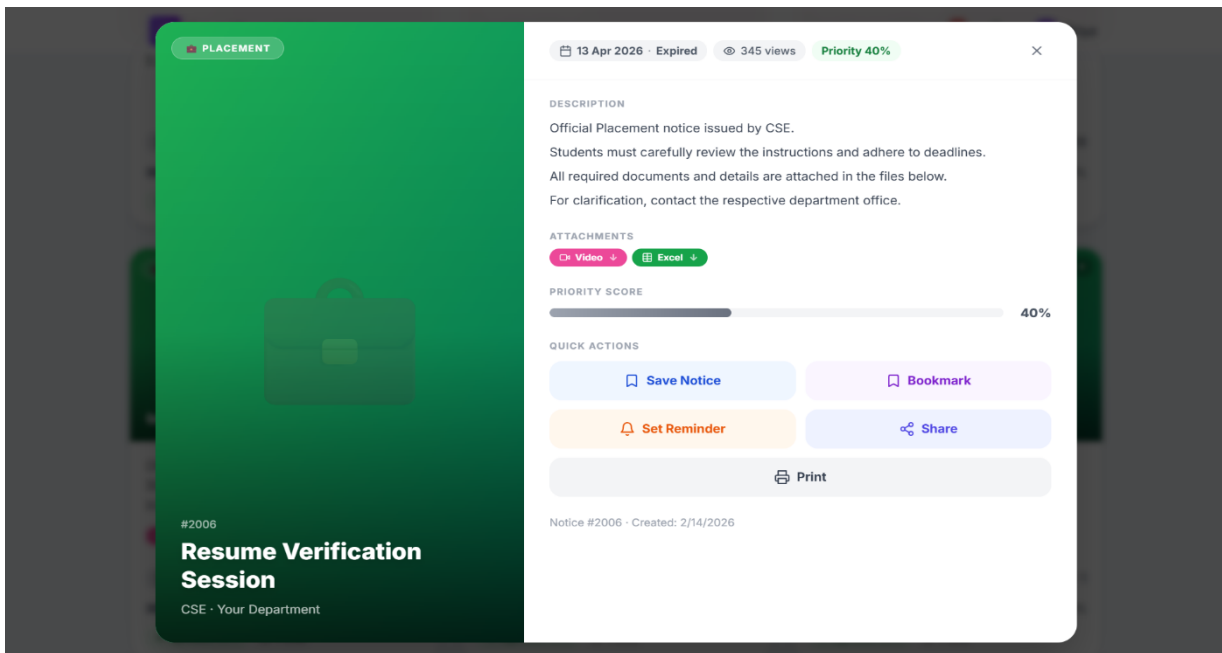


Figure 8: notice popup

STEP 5: Receiving an Alert When an administrator broadcasts an alert, the student receives an in-app notification badge on their dashboard. The student clicks the alert to view the severity level, message content, and the broadcasting scope (department-wide or institution-wide).

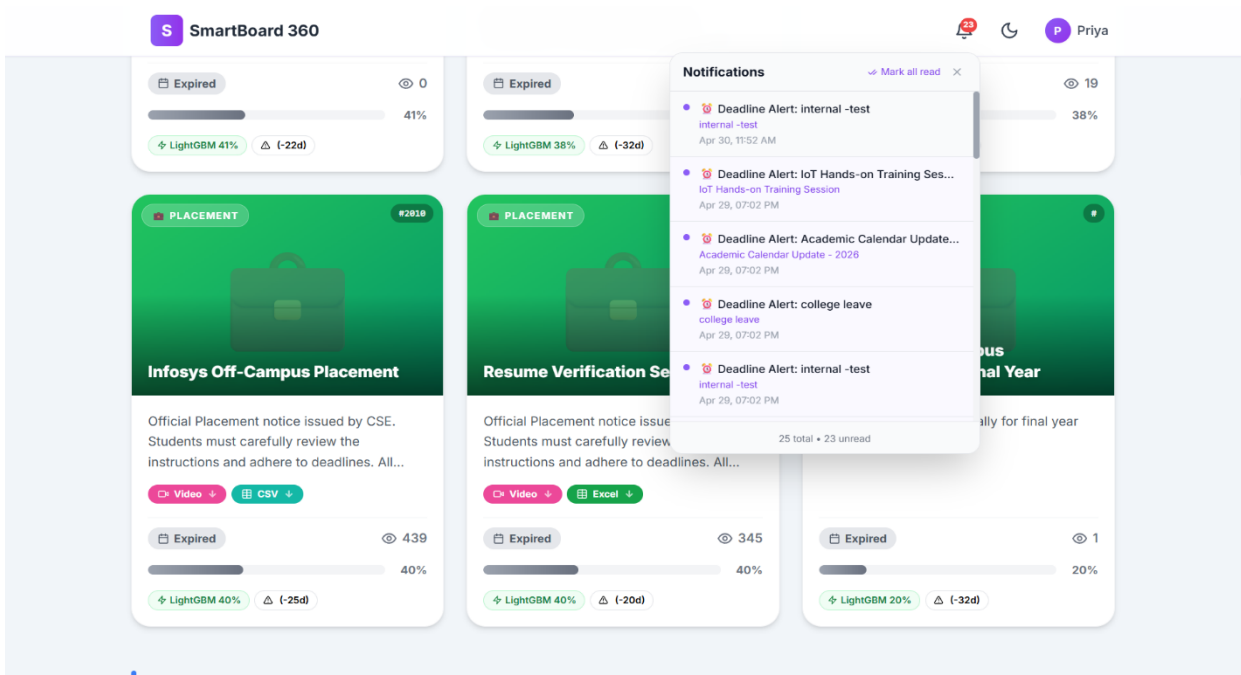


Figure 9: Student notification

STEP 6: Searching and Filtering Notices The student uses the search bar on the Notice Board to look up specific notices by keyword, category, or date range. Filter options allow the student to narrow results by department, priority level, or notice type — making it easy to find relevant announcements quickly.

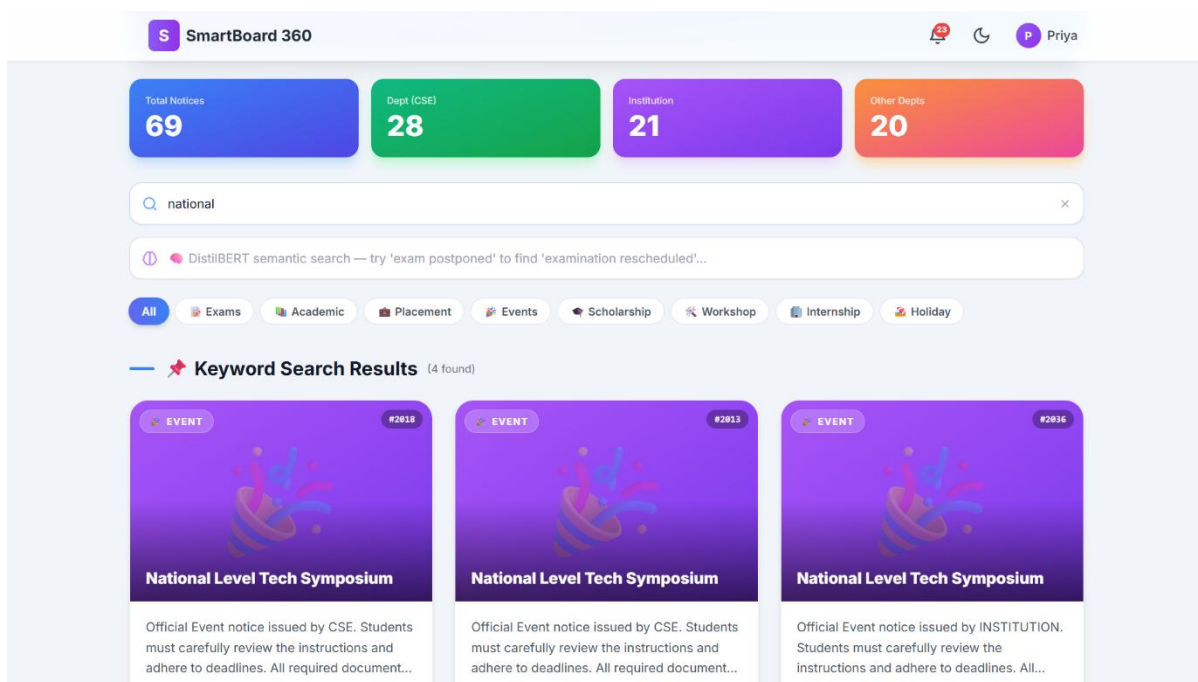


Figure 10: Searching

STEP 7: Bookmarking Important Email Reminder Notices The student clicks the bookmark icon on any important notice to save it for quick future reference. Then set the email reminder option.

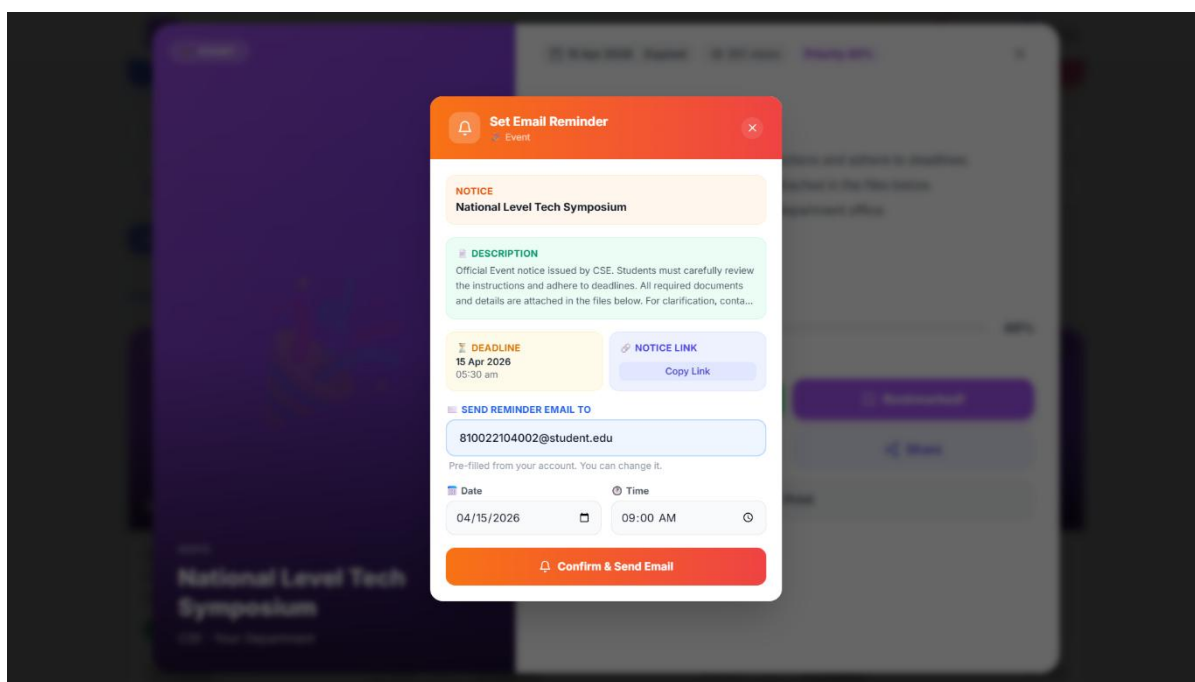


Figure 11: Set Reminder

STEP 8: Downloading Attached Documents The student opens a notice that contains an attached file such as a circular, syllabus, or official letter. The system displays a Download button alongside the notice content

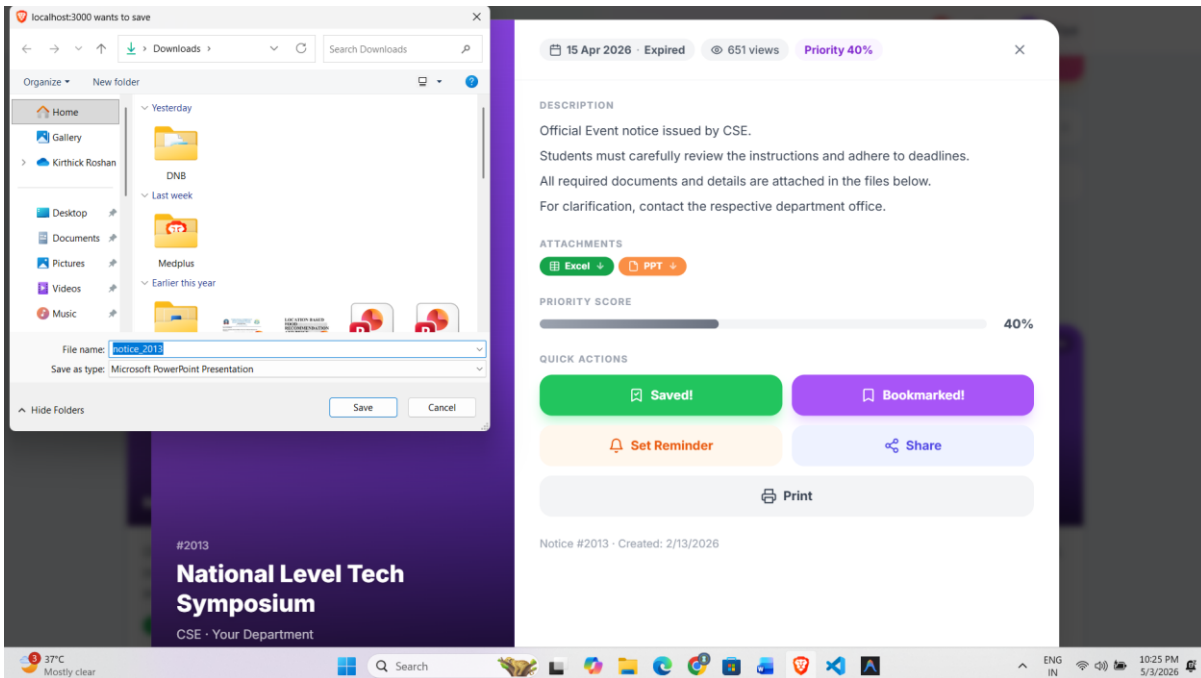


Figure 12: Downloading Attached Documents

STEP 9: Profile & Notification Preferences The student visits their profile page to update personal information and configure notification preferences — choosing which types of notices or alerts they wish to receive via email.

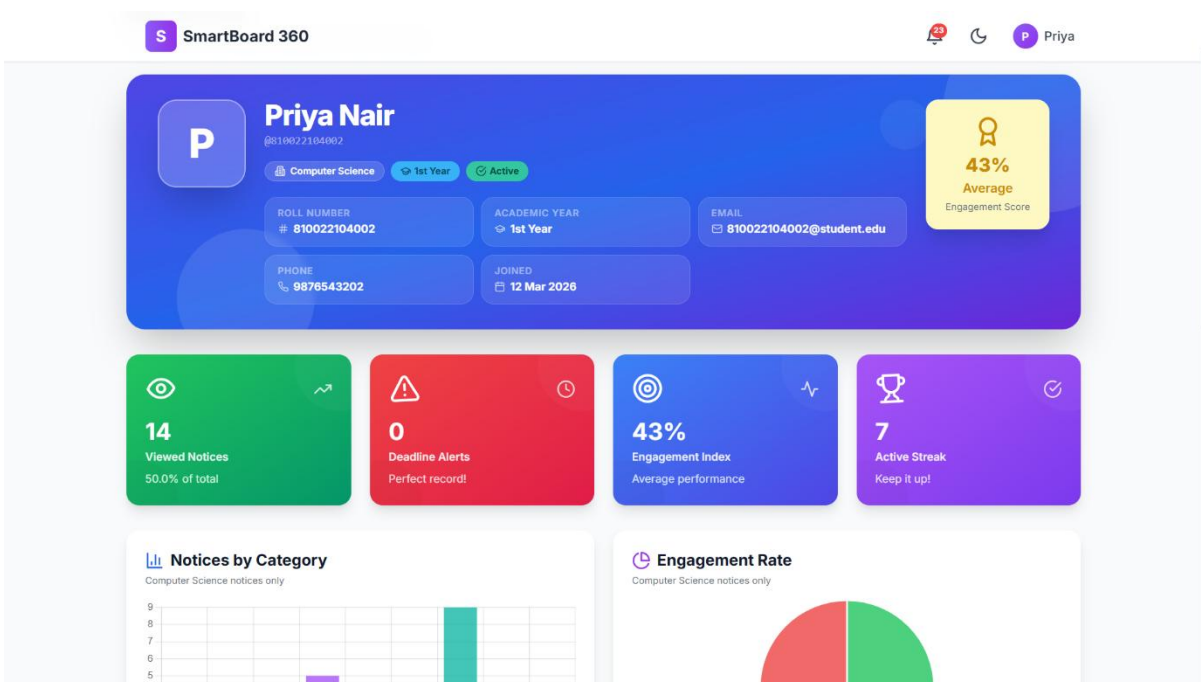


Figure 13: Profile page

STEP 10: Email Notification Received When a new timetable is published or a high-priority alert is broadcast, the student automatically receives an email notification at their registered address. The email contains the relevant notice or schedule details and a link to the system.

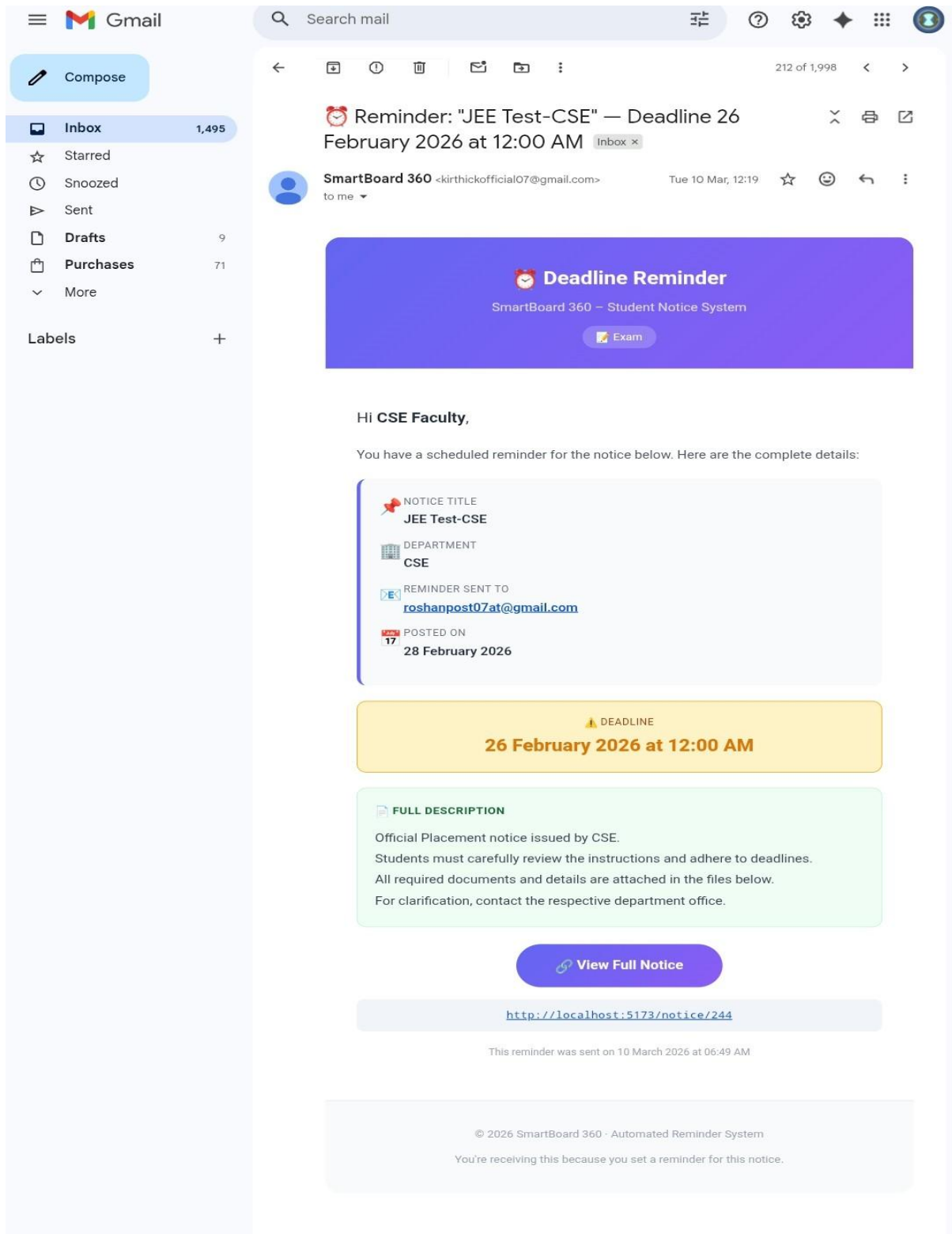


Figure 14: Email Notification Received

STEP 11: Admin Dashboard The administrator logs in and is directed to the Admin Dashboard, which displays an overview of total users, active notices, broadcast alerts, and department details. The admin can manage users, assign roles, monitor all notices and alerts across the institution, and perform quick actions such as broadcasting a system-wide alert.

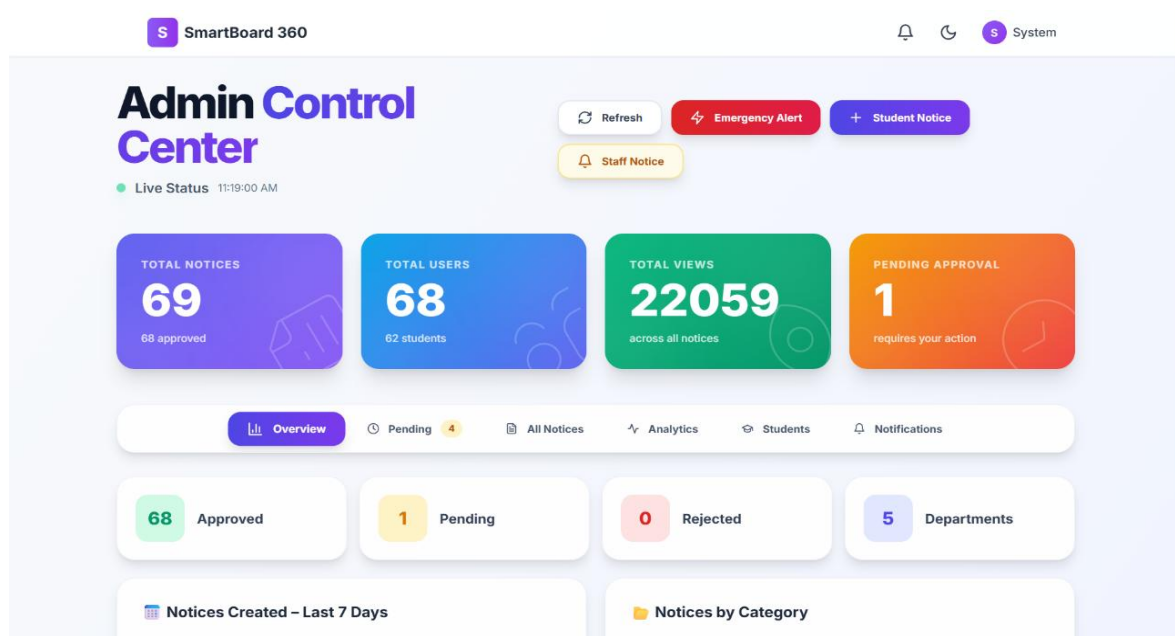


Figure 15: Admin Dashboard

STEP 12: Department Dashboard The department staff logs in and views the Department Dashboard, which shows notices, alerts, and student details specific to their department. Staff can publish new notices, manage timetable configurations, and track alert acknowledgements — all within their department scope.

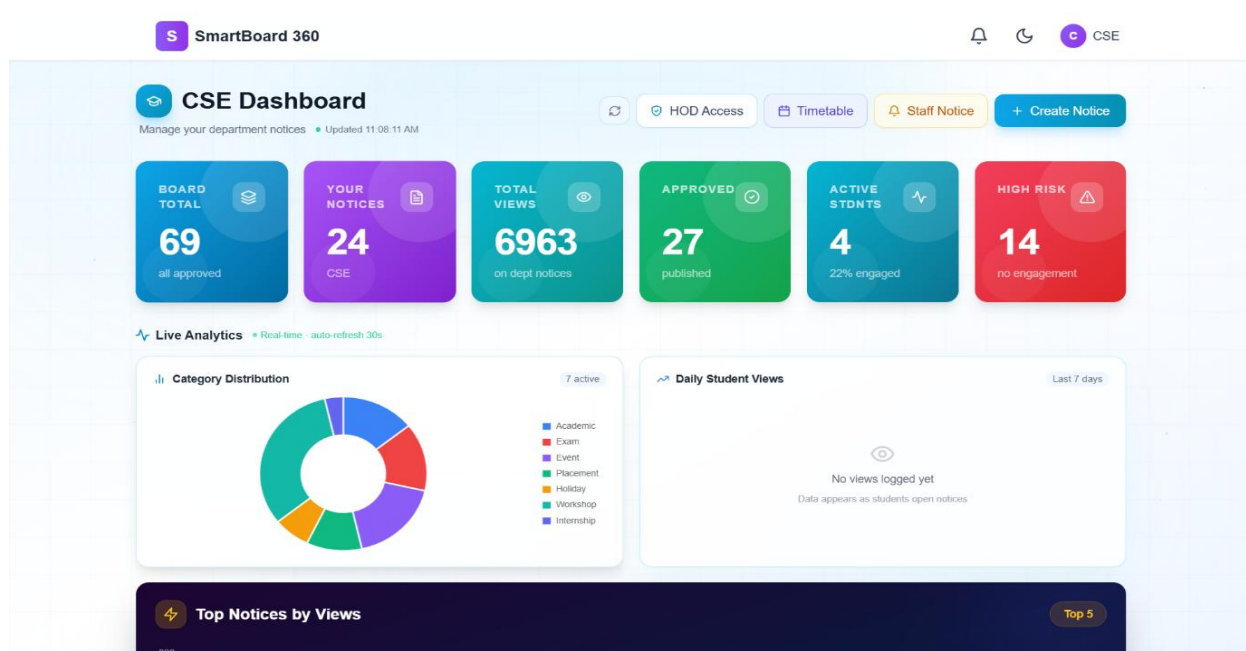


Figure 16: Department Dashboard

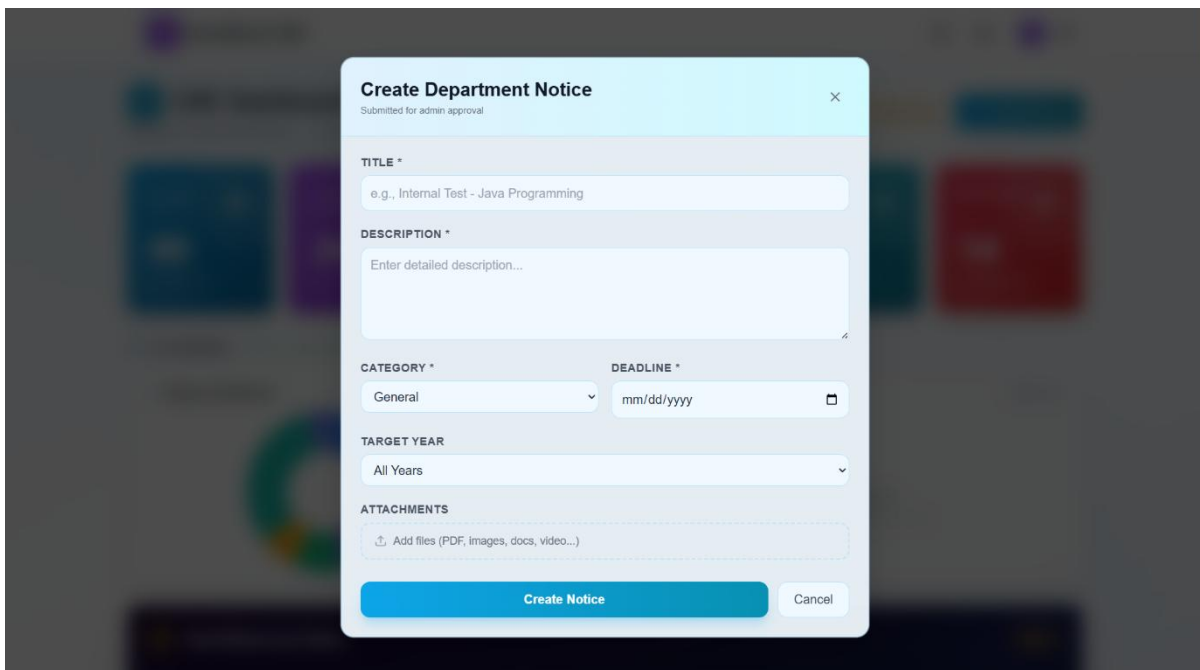


Figure 17: Create Notice

4.4 Results and Output

The implemented system was tested under various conditions to evaluate its performance and functionality. Users were able to log in, view role-filtered dashboards, and interact with notices and alerts without any issues. The timetable generation module successfully produced conflict-free schedules across multiple configurations. The email automation system delivered notifications accurately at the appropriate system events. The admin module handled user and department management efficiently. Overall, the system demonstrated reliability, accuracy, and ease of use, confirming the success of the implementation.

4.5 Performance Analysis

The performance of the Digital Notice Board (DNB) system is evaluated based on key parameters such as accuracy and efficiency, along with its ability to improve upon traditional notice management systems. These parameters help in assessing how effectively the system delivers notices, manages timetables, and ensures timely communication among users.

Accuracy is an important factor that determines how correctly the system manages and delivers notices, alerts, and timetables. The timetable generation algorithm is designed using constraint validation techniques, which ensure that there are no conflicts related to teachers, classrooms, or time slots. This guarantees a reliable and error-free timetable. Additionally, the system implements notice and alert filtering based on departments, ensuring that users receive only relevant and necessary information.

Role-Based Access Control (RBAC) further enhances accuracy by restricting data visibility according to user roles such as student, staff, department, Head of Department (HOD), and administrator. This ensures that each user accesses only the information they are authorized to view, thereby maintaining data integrity and correctness.

Accuracy Performance Analysis – DNB System

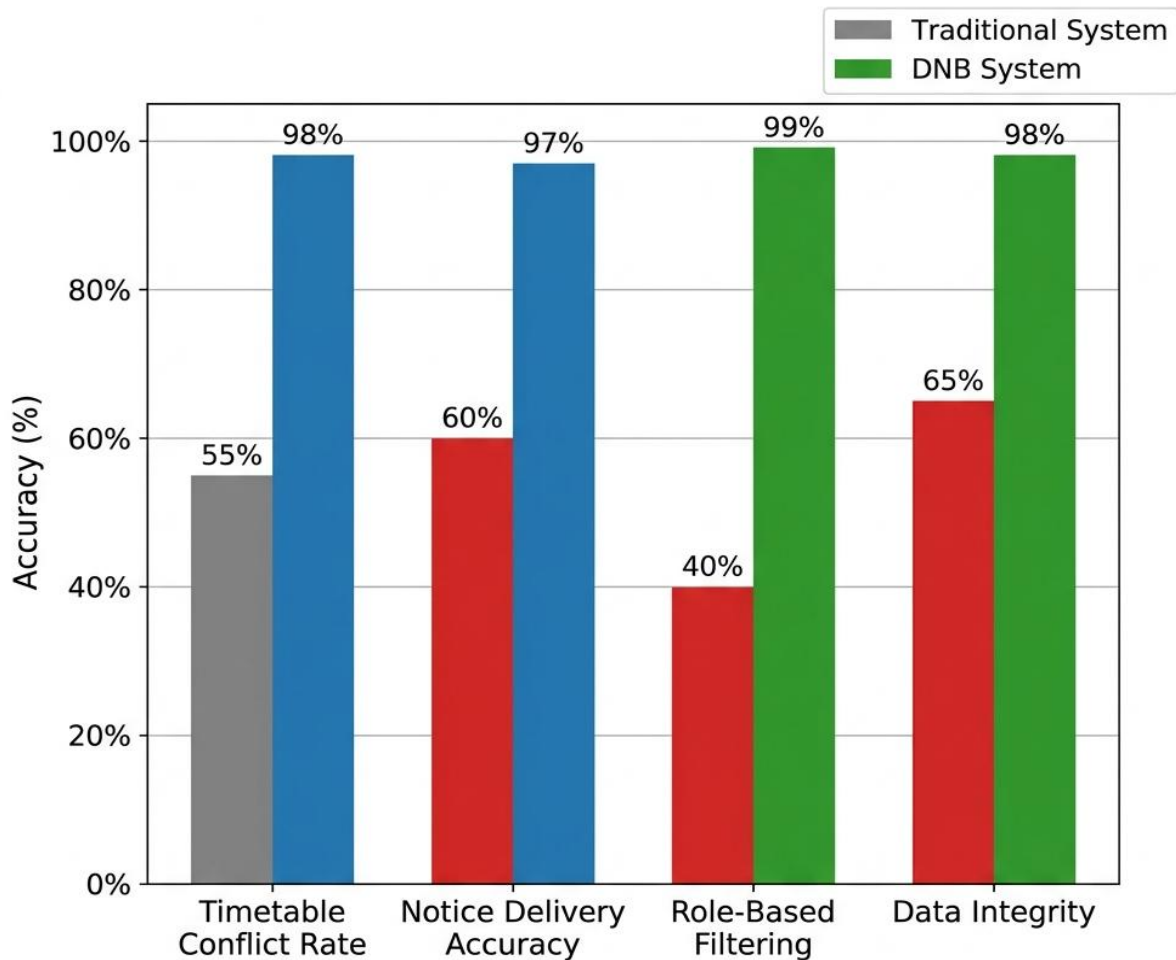


Figure 18 : Accuracy

Efficiency Measures how quickly and effectively the system performs its operations, including notice publishing, alert broadcasting, and timetable generation. The automated timetable generation feature significantly reduces the time required compared to manual scheduling, which traditionally takes several hours. The system also supports instant alert broadcasting, which minimizes communication delays and ensures that important updates reach users in real time. Email notification integration further improves efficiency by eliminating the need for manual follow-ups. Moreover, the use of optimized SQLite queries enables fast data retrieval, ensuring that the system responds quickly even when handling multiple requests. Overall, the DNB system demonstrates high performance by combining

accuracy and efficiency, making it a reliable and scalable solution for managing institutional communication and scheduling tasks.

Efficiency Performance Analysis – DNB System

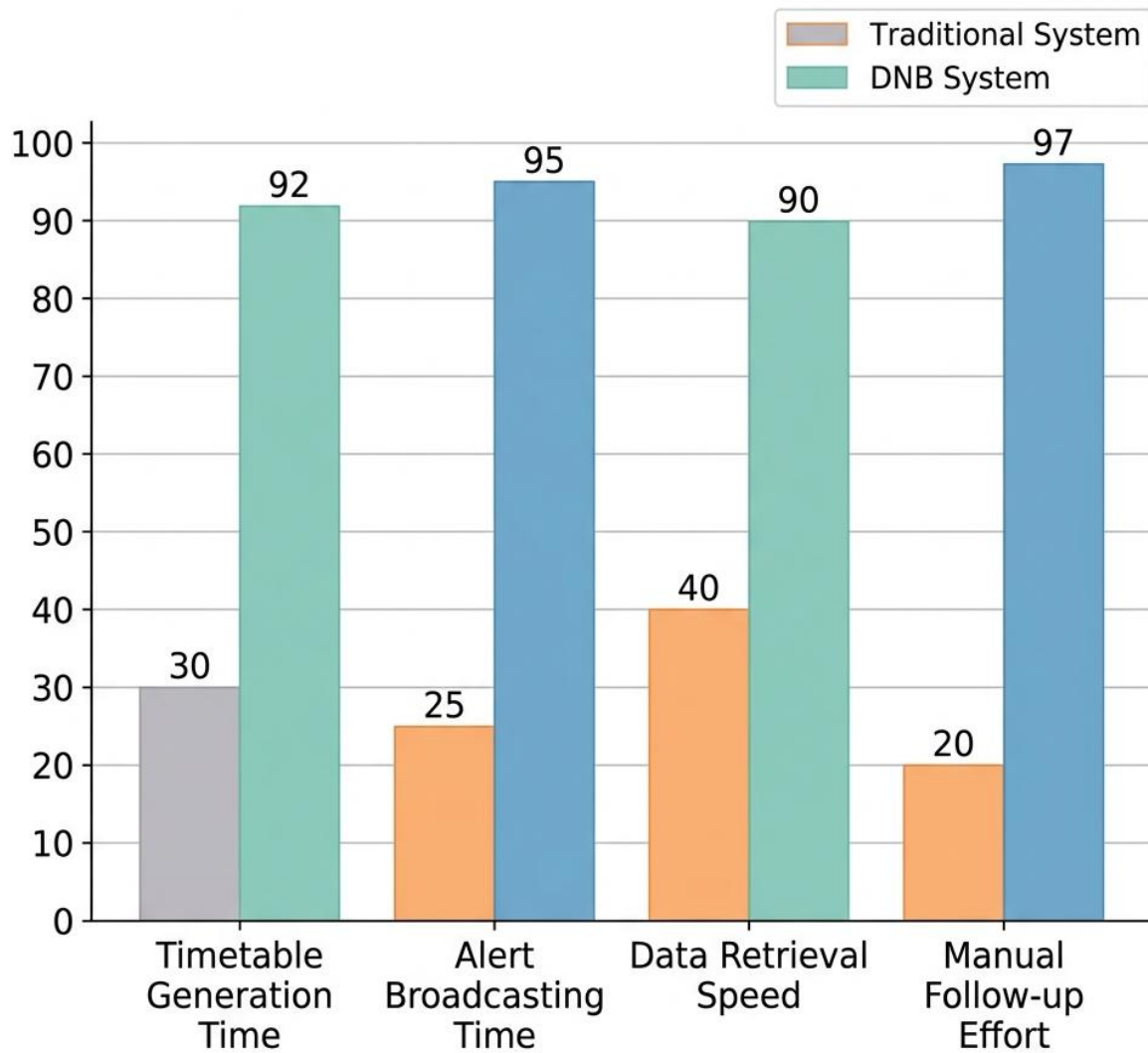


Figure 19: Efficiency

CHAPTER 5

CONCLUSION

5.1 Conclusion

The Digital Notice Board and Alert Management System (DNB) was developed to overcome the limitations of traditional notice distribution methods used in educational institutions. Conventional systems rely on physical boards and informal communication channels, which often lead to delayed updates and missed information. The proposed system provides a centralized and automated platform that ensures notices and alerts are delivered to the right users at the right time.

The system follows a three-tier architecture consisting of frontend, backend, and database layers, which improves scalability, maintainability, and performance. The frontend offers an easy-to-use interface for different users, while the backend handles business logic and workflow processing. The database ensures secure storage and efficient retrieval of data such as notices, user details, and activity logs.

A key feature of the system is role-based access control, where users such as students, staff, and administrators are provided with personalized dashboards. This helps in reducing information overload and ensures secure access to system functionalities. Additionally, the automated timetable generation module simplifies the scheduling process by generating conflict-free timetables, reducing manual effort and improving efficiency.

5.2 Future Enhancements

Although the Digital Notice Board and Alert Management System provides a comprehensive solution for institutional communication, there are several opportunities for further enhancement and expansion. Future improvements can focus on increasing scalability, improving user experience, and integrating advanced technologies to make the system more efficient and versatile. One key enhancement is the integration of real-time push notifications. Currently, alerts are delivered through in-app notifications and email. By implementing WebSocket-based or browser push notifications, users can receive instant updates even when the application is running in the background, thereby improving communication speed and reliability.

The system can also be enhanced through cloud deployment using platforms such as AWS or Microsoft Azure. Moving the backend and database to a cloud environment will improve scalability, data security, and system performance, especially when handling a large number of users across multiple institutions. Additionally, developing a dedicated mobile application for Android and iOS platforms will further improve accessibility. A mobile app can provide features such as push notifications, easy access to notices, and better user interaction, allowing users to stay updated anytime and anywhere.

5.3 Real World Extensions

The Digital Notice Board and Alert Management System can be further improved by incorporating several advanced features to enhance scalability, usability, and real-world applicability. One major enhancement is the migration to a cloud-based infrastructure such as AWS or Microsoft Azure, which enables high availability, automatic scaling, and secure data management. This would allow the system to handle a large number of users across multiple institutions efficiently.

Another important extension is the development of a dedicated mobile application for Android and iOS platforms. A mobile app can provide features such as push notifications, real-time alerts, and offline access to saved notices, improving user accessibility and engagement. Additionally, integrating the system with institutional platforms such as Learning Management Systems (LMS), Student Information Systems (SIS), and HR portals can automate notice generation based on academic or administrative events.

The system can also be enhanced by adding multi-channel notification support, including SMS and WhatsApp alerts, ensuring reliable communication even in low internet conditions. Furthermore, the platform can be extended to support multi-institution environments using a multi-tenant architecture, allowing different colleges to use the system independently under a single platform.

APPENDIX

Appendix A — System Configuration

The DNB system uses a .env file at the project root to manage all configuration values including the secret key, database path, and email settings. The .env.example file is included in the repository so that new developers can set up their local environment quickly. The DEBUG flag must be set to False before deployment.

env

SECRET_KEY=your-secret-key-here

DEBUG=True

DATABASE_URL=sqlite:///dnb.db

MAIL_SERVER=smtp.gmail.com

MAIL_PORT=587

MAIL_USE_TLS=True

MAIL_USERNAME=kirthickofficial071@gmail.com

MAIL_PASSWORD=majsxhssjangsh

To run the application, activate the virtual environment and start both the backend and frontend servers in separate terminals.

powershell

Backend

.\.venv\Scripts\Activate.ps1

cd backend

python app.py

Frontend

cd frontend

npm install

npm run dev

Appendix B — Database Schema

The DNB system uses SQLite 3 as the database. The schema includes tables for users, notices, alerts, attachments, and email logs. The role field in the users table controls access across all API endpoints.

sql

```
CREATE TABLE users (  
    id      INTEGER PRIMARY KEY AUTOINCREMENT,  
    full_name TEXT NOT NULL,  
    email   TEXT UNIQUE NOT NULL,  
    password TEXT NOT NULL,  
    role    TEXT CHECK(role IN ('STUDENT','DEPARTMENT','ADMIN')),  
    department TEXT NOT NULL,  
    year    TEXT  
);
```

```
CREATE TABLE notices (  
    id      INTEGER PRIMARY KEY AUTOINCREMENT,  
    title   TEXT NOT NULL,  
    content TEXT NOT NULL,  
    category TEXT NOT NULL,  
    department TEXT NOT NULL,  
    target_year TEXT DEFAULT 'ALL',  
    priority TEXT DEFAULT 'LOW',  
    status  TEXT DEFAULT 'PENDING',  
    expiry_date DATE,  
    created_by INTEGER REFERENCES users(id),  
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE alerts (  
    id      INTEGER PRIMARY KEY AUTOINCREMENT,
```

```

message TEXT NOT NULL,
severity TEXT NOT NULL,
scope TEXT DEFAULT 'ALL',
created_by INTEGER REFERENCES users(id),
created_at DATETIME DEFAULT CURRENT_TIMESTAMP);
CREATE TABLE email_logs (
id INTEGER PRIMARY KEY AUTOINCREMENT,
recipient TEXT NOT NULL,
subject TEXT NOT NULL,
status TEXT DEFAULT 'PENDING',
sent_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

```

Appendix C — REST API Reference

The DNB backend exposes a REST API under the /api path. All endpoints are role-protected and return JSON responses.

POST /api/auth/login — Authenticates the user and returns a token with role information.

json

```

{
  "token": "eyJ0eXAiOiJKV1Qi...",
  "user": {
    "id": 7,
    "full_name": "Roshan Kumar",
    "role": "STUDENT",
    "department": "CSE",
    "year": "2"
  }
}

```

GET /api/notices/ — Returns approved notices filtered by the user's department and year.

json

```
{
  "count": 2,
  "notices": [
    { "id": 42, "title": "Internal Assessment Schedule", "priority": "HIGH", "category":
"Exam" },
    { "id": 39, "title": "Campus Placement Drive", "priority": "URGENT", "category":
"Placement" }
  ]
}
```

POST /api/alerts/broadcast/ — Admin only. Broadcasts an alert to all or targeted users and triggers email notifications.

json

```
{
  "message": "College closed on 5th May.",
  "severity": "INFO",
  "scope": "ALL"
}
```

GET /api/timetable/?department=CSE — Returns the latest confirmed timetable for the specified department as a day-wise schedule grid.

json

```
{
  "department": "CSE",
  "schedule": {
    "Monday": [
      { "time": "09:00-10:00", "subject": "Data Structures", "teacher": "Dr. Rao", "room":
"Lab 3" }
    ]
  }
}
```

REFERENCES

1. Ahmed, K., Hassan, M., & Ibrahim, S., 2025, Smart Campus Communication Systems Using Cloud and IoT, Elsevier Journal of Smart Environments, pp. 45–58, Vol. 18 (Issue 2)
2. Ali, M., & Hassan, R., 2023, Machine Learning for Notification Prioritization, Elsevier Journal of Information Systems, pp. 12–25, Vol. 42 (Issue 1)
3. Bansal, A., & Kumar, N., 2023, IoT-Based Digital Notice Board Using Raspberry Pi, IEEE International Conference on Smart Systems, pp. 101–106, Proceedings (2023)
4. Chen, H., 2023, Matrix Factorization Techniques for Recommendation Systems, ACM Computing Surveys, pp. 1–34, Vol. 55 (Issue 4)
5. Das, S., & Roy, P., 2023, Smart Campus Communication Framework Using Web Technologies, Springer Journal of Smart Cities, pp. 233–245, Vol. 7 (Issue 3)
6. Garcia, F., 2024, AI-Based Personalization in Educational Information Systems, Elsevier Computers & Education, pp. 120–135, Vol. 195
7. Gupta, N., & Sharma, D., 2024, Context-Aware Notification Systems: A Survey, IEEE Access, pp. 55678–55695, Vol. 12
8. Gupta, V., 2023, Web-Based Centralized Information Systems for Universities, IEEE Conference on Cloud Computing, pp. 78–84, Proceedings (2023)
9. Joseph, A., & Thomas, J., 2024, Digital Communication Platforms in Smart Campuses, Springer International Journal of Educational Technology, pp. 89–102, Vol. 21 (Issue 2)
10. Khan, S., & Rehman, A., 2024, Predictive Analytics for Academic Deadline Alerts, Elsevier Journal of Applied Artificial Intelligence, pp. 310–325, Vol. 38 (Issue 5)
11. Kim, S., 2024, Hybrid Recommendation Systems for Personalized Alerts, ACM Transactions on Information Systems, pp. 1–20, Vol. 42 (Issue 1)
12. Li, X., & Chen, Y., 2025, Personalized Recommendation Models for Academic Notifications Using Machine Learning, Springer Journal of AI Research, pp. 210–228, Vol. 72
13. Lopez, M., 2024, Web-Based Smart Announcement Systems for Higher Education, IEEE Transactions on Learning Technologies, pp. 55–67, Vol. 17 (Issue 1)
14. Park, J., & Kim, H., 2025, Deadline-Aware Intelligent Reminder Systems Using Predictive Models, ACM Conference on Intelligent Systems, pp. 140–146, Proceedings (2025)
15. Patel, K., 2023, User Behavior Analysis for Smart Notification Systems, Elsevier Journal of Human-Computer Interaction, pp. 455–470, Vol. 39 (Issue 6)
16. Patel, R., & Shah, P., 2024, Smart University Framework Using IoT and Cloud Computing, Springer Journal of Smart Infrastructure, pp. 98–112, Vol. 10 (Issue 2)

- 17.Raj, M., & Kumar, V., 2025, Digital Transformation of Campus Notice Boards Using AI and Analytics, Springer Journal of Smart Education, pp. 65–80, Vol. 12
- 18.Roy, T., & Dutta, P., 2023, Deadline Prediction Using Survival Analysis Models, IEEE Transactions on Data Science, pp. 210–222, Vol. 6 (Issue 3)
- 19.Sharma, R., & Gupta, P., 2025, AI-Driven Smart Notification Systems for Educational Institutions, IEEE Access, pp. 12000–12015, Vol. 13
- 20.Singh, P., & Mehta, R., 2024, Student Engagement Through Intelligent Push Notifications, IEEE Transactions on Learning Technologies, pp. 200–215, Vol. 17 (Issue 2)
- 21.Singh, R., 2023, LightGBM for Structured Data Prediction, Springer Machine Learning Journal, pp. 890–905, Vol. 112 (Issue 5)
- 22.Verma, S., & Singh, A., 2025, Real-Time Event Notification Platform for Universities Using Web Technologies, IEEE International Conference on Web Engineering, pp. 55–60, Proceedings (2025)
- 23.Wang, L., & Zhao, X., 2024, Lightweight NLP Models for Notification Classification, ACM Conference on NLP Systems, pp. 300–306, Proceedings (2024)
- 24.Wong, T., & Lee, M., 2025, Machine Learning Based Priority Ranking of Notifications, IEEE Access, pp. 15000–15012, Vol. 13
- 25.Zhang, Y., Liu, Q., & Sun, J., 2025, Cloud-Based Smart Information Dissemination Platforms, Elsevier Future Generation Computer Systems, pp. 500–515, Vol. 158